Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ)
по направлению подготовки
09.04.01 Информатика и вычислительная техника

FINAL QUALIFICATION WORK
(MASTER'S THESIS)
Academic Program

09.04.01 Computer Science

Направленность (профиль) образовательной программы
«Искусственный интеллект и инженерия данных»
Field of Study:
«Artificial Intelligence and Data Engineering»

Тема **Нейросетевые алгоритмы суперразрешения для решения задачи фильтрации**

Topic **Super Resolution Neural Network Algorithms for Solving Filtration Problem**

Работу выполнил / *Prepared by*	**Лузинсан Анастасия / Luzinsan Anastassiya**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Конюхов Иван Владимирович / Ivan Konyukhov**	подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Tables

List of Figures

Abstract

Numerical simulation of fractured-porous reservoirs faces a trade-off between the accuracy of the fine-grid solution and the computational cost of building it. This thesis develops a neural network model that reconstructs fine-grid pressure and saturation fields from coarse-grid solutions produced by a simulator for the dual-porosity flow equations. Three residual super-resolution backbones are trained with a pixel-only loss and then fine-tuned under a combined objective that adds a domain-specific perceptual term, a reservoir-physics regularizer, and a relativistic adversarial term. Fine-tuning improves both the point-wise and the structural quality of the reconstruction at the same time, against the trade-off reported for natural-image super-resolution. The trained generator is exported to ONNX and integrated into a desktop application that compares the neural reconstruction with the output of the numerical solver and with stored reference solutions.

Chapter 1

Introduction

Global energy investment exceeded 3 trillion USD in 2024 [1], and the ability to forecast the performance of hydrocarbon assets remains a key factor of project viability. Numerical reservoir simulation serves as the foundational instrument for such forecasting. By constructing "digital twins" of the oil reservoir, engineers predict production dynamics and optimize recovery strategies over time horizons of decades. However, the application of reservoir simulation is inherently constrained by the complexity of the geological systems it attempts to represent. As noted by Saleri and Toronyi [2], simulation should not be viewed as a deterministic calculator but rather as a probabilistic tool subject to substantial uncertainties and non-uniqueness. The reliability of these models depends on their ability to capture the spatial distribution of rock properties and their impact on fluid flow [3]. Ringrose and Bentley emphasize that the primary purpose of a reservoir model is to provide predictive capacity by placing physical properties correctly in three-dimensional space, rather than merely generating visually appealing graphics.

Reservoir simulation, however, faces an intrinsic contradiction between pre-

dictive accuracy and computational cost. High-fidelity models, which are required to resolve fine-scale geological heterogeneities and complex multiphase flow phenomena, demand excessive computational resources. Coarse-grid models, although computationally efficient, often fail to capture essential fluid dynamics and produce systematic forecasting errors. The mismatch between the operational need for rapid, reliable forecasts and the limitations of classical simulators creates a substantial bottleneck in effective reservoir management.

To mitigate these uncertainties and maximize asset value, the industry has increasingly adopted "closed-loop reservoir management" [4]. This paradigm shifts reservoir management from a periodic activity to a near-continuous process, requiring the rapid evaluation of numerous scenarios to optimize well controls and update geological models in real-time. The practical implementation of this approach, however, varies significantly across the industry. One prevalent strategy, actively pursued by Russian operators such as Gazprom Neft, focuses on integrated digital platforms. As described by Kusimova et al. [5], machine learning in this context is primarily employed to automate peripheral processes surrounding the hydrodynamic simulator, such as interpreting seismic data or predicting well failures. While this enhances overall workflow efficiency, the core prediction engine remains a classical numerical simulator, and the cost of each individual high-fidelity forecast stays unchanged.

A contrasting philosophy is observed among Western majors, such as Equinor, which address geological uncertainty through ensemble modeling. Rather than seeking a single deterministic forecast, this strategy assesses a probabilistic range of outcomes using data assimilation methods like Ensemble Smoother with Multiple Data Assimilation (ES-MDA) [6]. Technologically, this approach necessitates launching thousands of parallel simulations to generate probabilistic production

profiles. The requirement to run such massive ensembles, while effective for uncertainty quantification, sits at the limit of current computational capabilities.

The computational cost of such ensemble-based workflows is prohibitive when applied to high-fidelity geological models. Hayder et al. [7] report that running models with billions of cells requires massive high-performance computing (HPC) infrastructure and can take weeks to complete a single simulation run. To make ensemble calculations feasible, practitioners typically upscale geological models, increasing grid block sizes from the required fine scale (e.g., 10×10 meters) to coarse scales (e.g., 100×100 meters) [8]. This upscaling process introduces a trade-off: while it reduces simulation runtime, it leads to the loss of information regarding fine-scale geological heterogeneities, such as high-permeability channels and flow barriers. Reviews of uncertainty quantification methods [9] attribute this loss to the inability of coarse models to resolve such features, resulting in numerical diffusion and a failure to reproduce complex, unstable flow phenomena like "viscous fingering".

Although reduced-order modeling (ROM) techniques, such as Trajectory Piecewise Linearization (TPWL), have been proposed to accelerate these calculations by factors of 100 or more [10], they often require computationally expensive offline training and may lack robustness when applied to highly nonlinear multiphase flow problems outside the training regime. Grid coarsening therefore remains the dominant strategy for practical ensemble modeling. However, recent studies by Pola et al. [11] demonstrate that this strategy is inadequate for complex recovery processes. The numerical dispersion induced by grid coarsening can introduce errors comparable in magnitude to those caused by an incorrectly configured physical model. Coarse grids fail to resolve micro- and meso-scale heterogeneities that govern the propagation of fluid fronts. From a mathematical

perspective, the dynamics of multiphase filtration in fractured-porous media are described by nonlinear parabolic partial differential equations, structurally analogous to the nonlinear diffusion problems investigated in recent super-resolution (SR) studies [12]. The resulting forecasts are systematically biased and cannot be corrected by simple parameter adjustment. Traditional deterministic reconstruction techniques, such as bilinear and bicubic interpolation, do not address this issue adequately either: recent benchmarks [12] show that these methods effectively act as low-pass filters, smoothing out sharp discontinuities and failing to recover the high-frequency geometric details of flow boundaries lost during coarsening. The field therefore lacks computational tools capable of recovering high-fidelity, physically consistent flow details from coarse-grid simulations without incurring the cost of fine-grid modeling. Deep learning, and SR algorithms in particular, offers a promising methodology for this problem by learning the mapping between large-scale dynamics and fine-scale physics.

Among the available descriptions of fractured-porous flow, which range from single-continuum effective-medium models to layered and multi-continuum formulations, this work adopts the dual-porosity model, because it explicitly separates the conductive fracture network from the storing matrix and thereby reproduces the sharp displacement fronts that motivate the reconstruction problem.

The goal of this thesis is to develop a software complex based on neural super-resolution for the accelerated reconstruction of transient pressure and saturation fields in fractured-porous media from coarse-grid simulations.

The research is organized around two research questions (RQ), each answered by a dedicated set of computational experiments:

- RQ1: Which of the residual SR architectures, trained with a pixel-wise loss,

delivers the lowest point-wise reconstruction error on the three-channel state fields of the reservoir?

- RQ2: Does fine-tuning these architectures within an adversarial framework alter the structural and spectral fidelity of the reconstruction, and what is the trade-off with point-wise accuracy?

The following hypotheses are formulated.

Hypothesis 1. Among residual SR architectures trained with a pixel-wise loss, a deeper architecture yields a lower point-wise reconstruction error than lighter architectures.

- Independent variable: residual SR architecture.
- Dependent variable: point-wise reconstruction error of the three-channel reservoir state field, whose channels are the global pressure, the fracture water saturation, and the matrix-block water saturation.
- Population: spatial distributions of multiphase flow state variables within the studied computational domain.
- Relation: increasing architectural depth raises model capacity and reduces the point-wise reconstruction error under a fixed loss function.

Hypothesis 2. Fine-tuning residual SR architectures within an adversarial framework increases the structural and spectral fidelity of the reconstruction and simultaneously decreases its point-wise accuracy.

- Independent variable: generator training scheme, pixel-wise or adversarial.

- Dependent variable: joint outcome across two metric groups, structural-spectral and point-wise.
- Population: spatial distributions of multiphase flow state variables within the studied computational domain.
- Relation: adversarial training shifts the reconstruction away from the conditional mean of the target distribution towards sharper states, which raises structural-spectral metrics and lowers point-wise metrics.

To achieve the goal and test the hypotheses, the following objectives are defined:

1. Conduct a domain analysis of the limitations of coarse-grid numerical modeling in fractured-porous media and review existing neural network approaches to spatial SR.
2. Implement and train three residual SR architectures with a pixel-wise loss to establish point-wise accuracy baselines.
3. Implement an adversarial training scheme and fine-tune the same three architectures under it to assess the change in structural and spectral fidelity of the reconstruction.
4. Conduct computational experiments comparing the obtained models with a metric suite covering point-wise accuracy, structural similarity, and physical consistency of the reconstructed fields.
5. Develop a prototype desktop application based on the PySide6 framework with integration to the simulator through a Google Remote Procedure Calls

(gRPC) interface for the practical demonstration of the selected model and for side-by-side comparison with classical interpolation and reference fine-grid solutions.

The practical significance of this work lies in reducing the computational cost of high-fidelity reservoir forecasting by partially replacing fine-grid simulation runs with neural SR applied to coarse-grid solutions. This enables larger ensemble sizes in uncertainty quantification workflows without a proportional increase in HPC requirements, making probabilistic decision-making more accessible in oil field development planning. The accompanying desktop application integrates with the existing simulation infrastructure through a gRPC interface and allows engineers to compare classical interpolation, neural SR, and reference fine-grid solutions in a unified visual environment. The numerical solver is implemented in C# for execution speed, while the training pipeline and the application are written in Python to reuse its mature deep-learning ecosystem and to allow rapid prototyping; the two sides communicate over a language-independent interface.

The thesis consists of an introduction, six chapters, a conclusion, a bibliography, and appendices. Chapter 2 presents a literature review covering reservoir geology, numerical simulation, and neural network methods. Chapter 3 details the research methodology, the experimental plan, and the architectural decisions. Chapter 4 describes the software implementation, including dataset generation and the training pipeline. Chapter 5 presents the results and their discussion, analyzing model quality and interpretability. Chapter 6 summarizes the main findings and outlines directions for further research.

Chapter 2

Literature Review

2.1 Fundamentals of Underground Fluid Dynamics

Reservoir management is a decision-making process under geological uncertainty. Da Cruz and Deutsch [13] note that development decisions, from well placement to production scheduling, rely on reservoir models whose calibration data come almost exclusively from wellbores. Wellbores sample a negligible fraction of the reservoir volume, so the reliability of production forecasts depends on how well the porous medium is characterized between the wells and on the accuracy with which the flow equations are applied.

A hydrocarbon reservoir is a porous geological formation that stores and transmits fluids. Two scalar fields define its state for any hydrodynamic calculation [14]: porosity ϕ, a dimensionless quantity that ranges from below 0.05 in tight formations to over 0.30 in clean sandstones, and absolute permeability, represented by a second-rank tensor K that describes the conductivity of the connected pore network. Porosity varies gradually in space, whereas permeability is heterogeneous and varies by orders of magnitude over short distances. Such vari-

ability reflects deterministic depositional processes rather than stochastic noise. A review of rule-based geological modeling [15] describes architectural elements such as high-permeability channels, shale drapes, and inclined heterolithic stratification. These structures are smaller than the typical 10-meter resolution of seismic surveys, and they control reservoir connectivity and sweep efficiency. A predictive model must resolve them to reproduce the non-linear dynamics of fluid displacement.

The structural architecture of a reservoir also controls the flow regime. A standard morphological classification [16] distinguishes layered reservoirs, in which properties vary vertically but stay roughly constant laterally and flow follows high-permeability layers, from naturally fractured reservoirs, which form a dual-porosity system: a low-permeability matrix that stores fluids and a high-permeability fracture network that conducts them. In a fractured-porous medium the macroscopic flow follows the connectivity of the fracture network, which produces non-linear flow paths and sharp displacement fronts that coarse-grid discretizations resolve poorly.

Following Kanevskaya, Kadet, and Selyakov [17], this work considers fractured-porous reservoirs in which porous matrix blocks are fully surrounded by fractures. The blocks do not form a continuous medium, and macroscopic filtration occurs only through the fracture network.

Multiphase flow in such systems is described by a single-level formulation under two assumptions: instantaneous saturation equilibrium within each matrix block, and local pressure equilibrium between a block and the surrounding fractures. The thermodynamic state of the system is then captured by three continuous scalar fields: a global pressure P, the water saturation S in the fracture network, and the water saturation $\bar{S}$ in the matrix blocks. Variables with an overbar denote

matrix-block quantities throughout this section.

Unlike in conventional porous media, the cross-flow between matrix blocks and fractures is driven mainly by elastic forces and not by capillary pressure. Its intensity q_Σ is proportional to the time derivative of the global pressure [17]:

$$q_\Sigma = -\bar{\beta}\frac{\partial P}{\partial \tau}, \tag{2.1}$$

where $\bar{\beta}$ is the elastic capacity coefficient of the matrix blocks and τ is continuous time.

Macroscopic transport through the fracture network follows an extension of Darcy's law. In this framework, the total filtration velocity is

$$\mathbf{V} = -\frac{KK^*}{\mu_1}\nabla P, \tag{2.2}$$

where K is the absolute permeability tensor of the fracture system, K^* is the effective phase mobility, and μ_1 is the reference fluid viscosity. Combining this convective flow with the elastic drainage from the matrix blocks gives the coupled system of non-linear partial differential equations [17]:

$$\beta\frac{\partial P}{\partial \tau} + \operatorname{div}\mathbf{V} = q_\Sigma, \tag{2.3}$$

$$m\frac{\partial S}{\partial \tau} + \beta_1^* S\frac{\partial P}{\partial \tau} + \operatorname{div}(f\mathbf{V}) = \lambda\ q_\Sigma, \tag{2.4}$$

$$\bar{m}\frac{\partial \bar{S}}{\partial \tau} + \bar{\beta}_1^*\bar{S}\frac{\partial P}{\partial \tau} = -\lambda\ q_\Sigma. \tag{2.5}$$

Here m and $\bar{m}$ are the effective porosities of the fractures and the matrix blocks, f is the fractional flow function of water in the macroscopic Darcy flow through the fractures, and λ is the fractional composition of the inter-porosity exchange

between matrix and fractures. Relative phase permeabilities describe how the mobility of each phase depends on local saturation; in the fractures they depend linearly on saturation, while in the matrix blocks they follow a higher-order power law governed by capillary effects [17].

The non-linear system above produces a hydrodynamic instability known as "viscous fingering", which is amplified in fractured-porous media: the fracture network provides preferential flow paths. The mobility ratio, defined as the ratio of the mobility of the displacing phase to that of the displaced phase, serves as the stability indicator: when it exceeds unity, small perturbations at the displacement front grow rather than decay [18]. A low-viscosity water phase displacing a high-viscosity oil therefore advances rapidly through the fracture network in irregular finger-like patterns. It bypasses oil trapped in the low-permeability matrix blocks and produces early water breakthrough at the producers. Resolving these fingers numerically requires a fine spatial discretization, since they are narrow features set by the local interplay of permeability and pressure gradients. Coarse grids reduce computational cost but introduce numerical diffusion that smears the sharp gradients and suppresses the instability, biasing recovery forecasts towards optimistic values. Accurate simulation of "viscous fingering" requires grid resolutions that are usually too fine to be feasible for full-field models.

2.2 Numerical Modeling

The coupled system in Equations 2.3–2.5 has no analytical solution for heterogeneous reservoirs, because of the non-linearity of relative permeability and the complexity of the boundary conditions. The industry therefore relies on numerical approximation, primarily the finite difference method (FDM) and the

finite volume method (FVM).

Peaceman [19] formalized the spatial discretization: in the block-centered formulation, petrophysical properties and state variables are averaged over the discrete volume of each grid block. Partial derivatives are then approximated by finite difference quotients obtained from Taylor expansions. For a scalar function $u(x)$, the central difference scheme gives

$$\frac{\partial u}{\partial x} \approx \frac{u(x+\Delta x) - u(x-\Delta x)}{2\Delta x}, \tag{2.6}$$

where Δx is the spatial grid spacing.

Discretization introduces a local truncation error ϵ_L. For a second-order central difference scheme [19],

$$\epsilon_L = \mathcal{O}(\Delta x^2) + \mathcal{O}(\Delta t), \tag{2.7}$$

where Δt is the time step. Accuracy is tied to grid resolution: halving the grid spacing cuts the approximation error by a factor of four and, in three dimensions, multiplies the number of cells N by eight. Since the cost of solving the resulting linear systems scales super-linearly in N, the cost of high-fidelity simulation becomes impractical at full-field scale.

Generalizations of this model add to the computational load. While the model above isolates macroscopic flow to the fracture network, generalized dual-permeability models [20], [21] describe simultaneous filtration in both the matrix and the fractures. The discrete pressure equation for these coupled systems is parabolic and is solved by implicit difference schemes. In the notation of

Konyukhov et al. [20] it takes the form

$$\mu_3(\alpha_T + \bar{\alpha}_T)\frac{\partial P}{\partial t} \approx \text{div}\,(\nabla P + \rho g), \tag{2.8}$$

where α_T and $\bar{\alpha}_T$ are the transport coefficients of the fracture and matrix systems that incorporate both absolute permeability and the effective phase mobility of each medium, μ_3 is the reference viscosity, ρ is the fluid density, and g is gravitational acceleration. The presence of two transport coefficients on the left-hand side captures the additional coupling that must be resolved when both media carry macroscopic flow.

The underlying trade-off between resolution and cost is unchanged by algorithmic improvements. Resolving fine-scale instabilities such as "viscous fingering" requires grid spacing finer than the scale of heterogeneity and leads to models with billions of cells. Coarsening the grid raises the truncation error and introduces artificial numerical diffusion. In fractured-porous media this diffusion smears the sharp saturation fronts in the fractures and dampens the local pressure gradients. Since the inter-porosity exchange in Equation 2.1 is driven by the time derivative of pressure, dampened gradients corrupt the mass-transfer calculation and degrade the recovery forecast. This is the trade-off addressed by the learning-based surrogate models examined in Section 2.4.

2.3 Performance Metrics for Flow Reconstruction

The quantitative assessment of surrogate models for multiphase reservoir flow requires more than a single point-wise score. Non-linear displacement fronts have sharp spatial profiles, and a metric that averages errors over the grid can

hide topological errors that change the engineering interpretation of the result. The literature on super-resolution and on remote sensing provides a layered set of metrics that target, in turn, point-wise accuracy, structural geometry, radiometric consistency across channels, the angular alignment of multi-channel signals, and physics-informed properties of the reconstructed field.

The Mean Absolute Error (MAE) and the Mean Squared Error (MSE) are the two basic point-wise measures. For a reference field X and a predicted field Y defined on N grid cells,

$$\mathrm{MAE}(X,Y) = \frac{1}{N}\sum_{i=1}^{N} \mid x_i - y_i \mid, \qquad \mathrm{MSE}(X,Y) = \frac{1}{N}\sum_{i=1}^{N}(x_i - y_i)^2. \tag{2.9}$$

Mathieu et al. [22] argued that MSE is biased towards spatially averaged solutions, and that MAE, being less sensitive to large outliers, tends to preserve edges better in dense prediction tasks. The Peak Signal-to-Noise Ratio (PSNR) rewrites MSE on a logarithmic scale relative to the dynamic range of the reference signal [23]:

$$\mathrm{PSNR}(X,Y) = 10 \cdot \log_{10}\left(\frac{R(X)^2}{\mathrm{MSE}(X,Y)}\right), \tag{2.10}$$

where $R(X) = \max(X) - \min(X)$ is the dynamic range of the reference field X. The dynamic range adapts the metric to the actual span of values in each channel, which is needed when channels follow different normalization schemes or have different numerical ranges. PSNR is reported per channel and as a channel average. A complementary indicator of robustness is the maximum absolute error, $\max_i \mid x_i - y_i \mid$, which characterizes the worst-case point deviation and is sensitive to localized non-physical spikes.

Point-wise metrics describe per-cell deviations but do not capture the geome-

try of fluid fronts or the connectivity of swept regions. Wang et al. [24] introduced the Structural Similarity Index (SSIM) to separate the similarity assessment from per-pixel intensity comparison. SSIM decomposes the comparison into three statistical components computed over a local sliding window: a luminance term l, a contrast term c, and a structural cross-correlation term s,

$$l(x,y) = \frac{2\,\mu_x\,\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}, \quad c(x,y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}, \quad s(x,y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3}, \tag{2.11}$$

with local means μ_x, μ_y, standard deviations σ_x, σ_y, cross-covariance σ_{xy}, and small numerical constants C_1, C_2, C_3 that stabilize the division near zero. The composite index used in practice combines luminance and the joint contrast-structure term,

$$\mathrm{SSIM}(x,y) = \frac{(2\,\mu_x\,\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{2.12}$$

and takes values in $[-1, 1]$, with 1 corresponding to perfect structural identity. SSIM penalizes the artificial smoothing of sharp fronts and the geometric misalignment of flow barriers more heavily than uniform offsets, which makes it informative for fields that contain narrow displacement structures.

Multi-channel physical states require evaluation criteria that respect the joint structure of the channels. Wald [25] introduced the Error Relative Global Adimensional Synthesis (ERGAS) in the context of multispectral image fusion. For a reference field X and a predicted field Y with C channels,

$$\mathrm{ERGAS}(X,Y) = 100\,\frac{h}{l}\,\sqrt{\frac{1}{C}\sum_{c=1}^{C}\frac{\mathrm{MSE}_c}{\mu_c^2}}, \tag{2.13}$$

where h/l is the spatial resolution ratio between the high- and low-resolution

domains, MSE_c is the mean squared error of the c-th channel, and μ_c is the mean of the c-th reference channel. The per-channel normalization by μ_c^2 makes ERGAS appropriate when channels span different numerical ranges, as for coupled pressure and saturation fields. Kruse et al. [26] introduced a complementary criterion, the Spectral Angle Mapper (SAM), which evaluates the preservation of inter-channel relationships by computing the angle between the reference and predicted channel vectors at each spatial location,

$$\mathrm{SAM}(x, y) = \arccos\left(\frac{\sum_{c=1}^{C} x_c y_c}{\sqrt{\sum_{c=1}^{C} x_c^2}\,\sqrt{\sum_{c=1}^{C} y_c^2}}\right). \tag{2.14}$$

A low SAM value indicates that the surrogate model preserves the joint statistics across channels rather than reconstructing them independently. For a dual-porosity formulation, in which pressure, fracture saturation, and matrix saturation are coupled through the inter-porosity exchange term of Section 2.1, a low SAM corresponds to a reconstruction that respects the coupling and not merely the marginal distribution of each channel.

A separate family of metrics, originating in image restoration, targets the geometry of edges and fronts rather than the global statistics of the field. Spatial gradients computed with the orthogonal Sobel operators [27] approximate these local edge structures, and Mathieu et al. [22] formulated image-prediction objectives based directly on such gradients. A gradient-consistency criterion, here termed the Gradient Mean Absolute Error (GMAE), builds on these Sobel-based objectives and compares the magnitudes of the Sobel gradients of the reference

field X and the reconstructed field Y:

$$\mathrm{GMAE}(X,Y) = \frac{1}{N}\sum_{i=1}^{N}\Big|\,|\,\nabla X\,|_i - |\,\nabla Y\,|_i\,\Big|, \tag{2.15}$$

where $|\,\nabla X\,|$ denotes the per-pixel magnitude of the Sobel gradient of X computed separately for each channel, and the sum runs over all channel-pixel entries with N their total count. Unlike MSE, this criterion penalizes the smearing of fronts but tolerates small spatial misalignments, which makes it sensitive to the recovery of front sharpness rather than to absolute front position.

A further family of criteria embeds physical priors directly into the evaluation. Raissi et al. [28] formalized this idea in the physics-informed neural network framework, where residuals of the governing partial differential equation are used as a training signal that constrains the network output to obey the underlying physics. The same principle applies to evaluation: for diffusive fields governed by parabolic equations, such as the pressure field in the system of Equations 2.3–2.5, the magnitude of a discrete Laplacian operator on the reconstructed field measures the deviation from the smoothness implied by the governing equation. Combined with the gradient-based criterion above, such physics-aware indicators provide a domain-specific layer of evaluation that complements purely statistical metrics by reporting to what extent a reconstruction departs from properties enforced by the governing equations.

2.4 Evolution of Super-Resolution Models

Reconstruction of fine-scale spatial fields from coarse-grid approximations is a classical inverse problem in computer vision, where it is studied under the

name of single-image super-resolution (SR). The time-dependent nature of a reservoir simulation is compatible with this formulation: at any fixed simulation step the task reduces to recovering high-frequency spatial detail from a coarse two-dimensional snapshot, and the temporal evolution is recovered by applying the same reconstruction at each step. The literature on super-resolution architectures therefore provides directly applicable building blocks. Convolutional neural networks are the dominant family in this setting, because the sequential application of learnable convolution kernels extracts hierarchical features while preserving the local structure of the input.

2.4.1 Convolutional and Residual Networks

Before the introduction of deep learning, spatial upscaling relied almost exclusively on deterministic interpolation methods such as bilinear or bicubic filtering. These linear low-pass filters cannot generate new high-frequency content. They smear sharp gradients in the reconstructed signal and produce artifacts wherever the signal departs from the local smoothness assumed by the interpolation kernel.

Dong et al. [29] introduced the Super-Resolution Convolutional Neural Network (SRCNN), which reinterprets the sparse-coding pipeline as a three-layer convolutional network and learns the mapping from a low-resolution (LR) patch space to a high-resolution (HR) patch space end-to-end. The three layers correspond to patch extraction and representation with 9×9 kernels, non-linear feature mapping with 1×1 or 5×5 convolutions, and final reconstruction. SRCNN outperformed classical interpolation, but it requires the input to be upsampled to the target resolution by bicubic interpolation before entering the network. As a

result, all convolutions operate in the high-resolution space, the computational cost grows quadratically with the upscaling factor, and the practical depth of the network is limited to a small number of layers with a short receptive field.

Shi et al. [30] introduced sub-pixel convolution, often referred to as the PixelShuffle operator, as an alternative to pre-upsampling. All feature extraction is performed in the low-resolution space, and the final layer learns a set of upscaling filters together with a periodic shuffling operator that rearranges a multi-channel low-resolution tensor into a single high-resolution output. Moving the main computation to the low-resolution space reduces memory consumption and inference time, and post-upsampling has since become the standard organization of super-resolution networks.

Increasing the depth of the network expands the receptive field and the representational capacity, but stacking plain convolutional layers introduces the degradation problem, in which training error first saturates and then increases. He et al. [31] addressed this with the residual block. Let $H(x)$ denote the underlying mapping that a stack of layers is expected to fit; the residual block parameterizes the residual $F(x) := H(x) - x$ and reconstructs the original mapping as $F(x) + x$ through an identity shortcut that bypasses one or more convolutional layers. The additive shortcut allows gradients to propagate through the network during backpropagation, mitigates the vanishing gradient problem, and stabilizes the training of very deep models. In super-resolution architectures, residual blocks are typically used without downsampling so that the spatial resolution of feature maps is preserved throughout the body of the network.

Kim et al. [32] applied this idea at the level of the whole network with the Very Deep Super-Resolution model (VDSR). VDSR stacks twenty convolutional layers and predicts only the global residual between the bicubically upsampled

low-resolution input and the high-resolution target, an approach known as global residual learning. The residual formulation expanded the receptive field from 13×13 to 41×41 grid cells and let the model exploit a larger spatial context. Kim et al. also introduced adjustable gradient clipping, which permitted learning rates four orders of magnitude higher than those used in shallower predecessors, and they applied zero-padding before every convolution to keep the spatial dimensions of feature maps unchanged through the forward pass. VDSR, however, kept the pre-upsampling strategy of SRCNN and inherited its quadratic dependence of cost on the upscaling factor.

Ledig et al. [33] combined sub-pixel post-upsampling with a residual body in the SRResNet generator of the SRGAN framework. SRResNet stacks residual blocks in the low-resolution feature space, applies sub-pixel convolution at the network tail, and uses Batch Normalization inside each residual block to stabilize training. SRResNet became a reference architecture for subsequent residual super-resolution work and the starting point for several refinements.

Lim et al. [34] refined the SRResNet design in two variants: the Enhanced Deep Super-Resolution model (EDSR), a single-scale model with a single output branch, and the Multi-scale Deep Super-Resolution model (MDSR), in which a shared residual body is preceded and followed by scale-specific modules so that several upscaling factors can be trained jointly. Two architectural choices distinguish this family from SRResNet. First, Batch Normalization is removed from the residual block. Batch Normalization standardizes feature maps using mini-batch statistics, which is useful for classification but distorts the absolute range of values that super-resolution networks must reconstruct. Removing it preserves the relative magnitudes of the signal and reduces memory consumption. Second, the output of each residual block is multiplied by a small constant,

the residual scaling factor, with a value of 0.1 in the original paper. Residual scaling stabilizes the training of very deep unnormalized networks by keeping the magnitude of the residual contribution bounded. A direct variant of the SRResNet design that adopts the same removal of Batch Normalization, commonly referred to as MSRResNet, is widely used as a regression baseline in later generative super-resolution work, including the Enhanced Super-Resolution Generative Adversarial Network (ESRGAN) family discussed in Section 2.4.2.

A separate line of work targets parameter efficiency rather than depth. Hui et al. [35] proposed the Information Multi-Distillation Network (IMDN), which splits the feature map at each block into a distilled portion that is preserved for later aggregation and a refined portion that undergoes further convolutions, so that information from earlier layers is preserved without redundant computation. Liu et al. [36] refined this design as the Residual Feature Distillation Network (RFDN). Each residual feature distillation block in RFDN passes the input through several shallow distillation paths, projects the distilled features with 1×1 convolutions, concatenates them, and feeds the result through an enhanced spatial attention module that re-weights the spatial locations of the feature map. The distillation structure achieves reconstruction quality comparable to that of much deeper residual networks at a fraction of the parameter count.

The architectures described above are trained with point-wise pixel losses such as Mean Squared Error or Mean Absolute Error. Minimizing the average per-cell error drives the network toward the conditional mean of the high-resolution target given the low-resolution input. The conditional mean is a smooth function of the input even when the underlying distribution of high-resolution targets contains sharp edges and fine texture, so models trained in this way systematically lose high-frequency content. This effect, known as regression to the mean, is the central

limitation that motivates the generative formulations examined in Section 2.4.2.

2.4.2 Generative Adversarial Frameworks

The regression-to-the-mean effect described in Section 2.4.1 can be stated formally. For a generator G_θ trained by minimizing the expected squared error between $G_\theta(X)$ and the high-resolution target Y, classical estimation theory shows that the optimum is the conditional expectation:

$$G_\theta^*(X) = \mathbb{E}[Y \mid X]. \tag{2.16}$$

When several plausible high-resolution outputs are consistent with a given low-resolution input, this optimum is a deterministic spatial average and cannot reproduce the sharp features of any individual realization. Generative adversarial networks (GAN) were introduced to optimize a different objective, one that matches the distribution of outputs to the distribution of high-resolution targets rather than the conditional mean.

Goodfellow et al. [37] formulated the generative adversarial framework as a minimax game between a generator G_{θ_G} that produces samples and a discriminator D_{θ_D} that estimates the probability that a given sample comes from the data distribution rather than the generator. Ledig et al. [33] adapted this framework to single-image super-resolution in the Super-Resolution Generative Adversarial Network (SRGAN) by conditioning the generator on the low-resolution input I^{LR}. The resulting objective is

$$\min_{\theta_G} \max_{\theta_D} \; \mathbb{E}_{I^{HR}}\big[\log D_{\theta_D}(I^{HR})\big] + \mathbb{E}_{I^{LR}}\big[\log\big(1 - D_{\theta_D}(G_{\theta_G}(I^{LR}))\big)\big]. \tag{2.17}$$

The discriminator acts as a learned loss function: instead of penalizing the generator for deviating from a fixed point-wise target, it penalizes the generator for producing outputs that are distinguishable from samples drawn from the data distribution.

Alongside the adversarial loss, SRGAN introduced a perceptual loss to compare the generator output and the target in the activation space of a pre-trained classifier rather than in the pixel space. Let ϕ_j denote the feature map produced by the j-th layer of a VGG network applied to its input, with spatial dimensions $W_j \times H_j$. The perceptual loss is

$$\mathcal{L}_{\text{VGG}} = \frac{1}{W_j H_j} \left\| \phi_j(I^{HR}) - \phi_j(G_{\theta_G}(I^{LR})) \right\|_2^2. \tag{2.18}$$

Comparison in feature space is less sensitive to small spatial misalignments than per-pixel error and rewards the recovery of textures and edges even when point-wise metrics deteriorate.

Wang et al. [38] refined SRGAN with three modifications. First, the residual block of the generator is replaced by a Residual-in-Residual Dense Block (RRDB), a nested structure that combines dense connections with residual scaling and increases the capacity of the generator through wider connectivity rather than deeper stacking. Second, Batch Normalization is removed from the generator for the same reasons as in EDSR. Third, the standard discriminator is replaced by the Relativistic average Discriminator of Jolicoeur-Martineau [39]. Rather than predicting the absolute probability that a given input is real, the relativistic discriminator predicts whether one input is more realistic than the other on average:

$$D_{Ra}(I^{HR}, I^{SR}) = \sigma\big(C(I^{HR}) - \mathbb{E}_{I^{SR}}[C(I^{SR})]\big), \tag{2.19}$$

where C is the pre-sigmoid output of the discriminator, σ is the sigmoid activation, and a symmetric expression defines $D_{Ra}(I^{SR}, I^{HR})$. The relativistic formulation produces stronger gradients for the generator and is reported to reduce mode collapse in adversarial training.

Several auxiliary techniques have become standard for stabilizing adversarial training. Miyato et al. [40] proposed spectral normalization, which constrains the Lipschitz constant of each layer of the discriminator by dividing its weights by the largest singular value computed by power iteration. Spectral normalization controls the rate at which the discriminator output can change with respect to its input and has become a standard stabilization technique for GAN discriminators.

The adversarial framework improves the recovery of high-frequency content but is also known to produce sample-level artifacts that the discriminator alone does not penalize, since the adversarial objective constrains the distribution of outputs rather than the fidelity of any individual reconstruction. Additional constraints are therefore needed to keep the generated samples close to plausible reconstructions, through physics-aware regularization terms acting on the generator output, examined in Section 2.5.

2.4.3 Spatiotemporal and Attention-Based Alternatives

Beyond convolutional and generative-adversarial architectures, three other families of models have been applied to spatial reconstruction: spatiotemporal recurrent networks, attention-based Transformer models, and denoising diffusion models.

Convolutional Long Short-Term Memory (ConvLSTM) networks were introduced by Shi et al. [41] to model spatiotemporal correlations through convo-

lutional state transitions. ConvLSTM replaces the dense matrix multiplications of a standard Long Short-Term Memory (LSTM) cell with convolutions, so that the recurrent state preserves a two-dimensional spatial structure. ConvLSTM is applied to tasks in which prediction at each step depends on a hidden temporal state that carries information across the sequence. Autoregressive inference over long sequences accumulates approximation error at each step, and the resulting drift limits the prediction horizon.

Transformer-based super-resolution models, exemplified by the SwinIR network of Liang et al. [42], replace localized convolution by self-attention over windows of spatial tokens. Self-attention captures long-range dependencies with fewer intermediate layers than stacks of convolutions. The computational cost of attention scales quadratically with the number of tokens within each window, and the dense projections in the Transformer embedding generally yield a higher per-layer parameter count and inference cost than convolutional architectures with a comparable receptive field.

Denoising diffusion models [43], [44] generate samples by reversing a learned noise process over many iterations. They have produced competitive results for both unconditional image synthesis and conditional super-resolution and offer strong distributional coverage in the sense that the family of generated samples reflects the variability of the target distribution. The principal cost of diffusion is the iterative sampling procedure: producing one high-resolution sample requires repeated evaluation of the denoising network, which raises the inference cost by an order of magnitude or more compared to single-pass feedforward generators.

2.5 Super-Resolution in Reservoir Engineering

The application of super-resolution to underground fluid mechanics differs from standard computer vision in the nature of the input data. Standard super-resolution assumes that the low-resolution input is obtained by applying a deterministic blurring and downsampling operator to a high-resolution original. In computational physics, the low-resolution input is a numerical solution of the governing equations on a coarse mesh; it contains numerical diffusion and omits subgrid-scale phenomena. The mapping that the network must learn therefore recovers physical information lost during coarse-grid simulation rather than inverting a known blurring kernel.

Deng et al. [45] demonstrated the use of generative adversarial networks for the reconstruction of turbulent velocity fields and showed that adversarial training restores the high-frequency component of the energy spectrum that conventional regression models suppress. To characterize the recovery of dominant eddies, the authors used the two-point spatial correlation coefficient of velocity components,

$$R_{vv}(x, y, x_0, y_0) = \frac{\langle v(x, y) v(x_0, y_0) \rangle}{v_{rms}(x, y)\, v_{rms}(x_0, y_0)}, \tag{2.20}$$

where the angle brackets denote an ensemble average over realizations. They reported that conventional regression models act as low-pass filters and suppress small-scale fluctuations, while the adversarial framework recovers high-wavenumber content. The recovered spectral content corresponds to sharp boundaries and small-scale instabilities such as “viscous fingering”, which point-wise regression cannot reproduce.

Several studies have proposed hybrid architectures that combine adversarial

training with explicit physical regularization. Doloi et al. [46] developed a super-resolution constrained generative adversarial network for upscaling oil saturation maps in single-porosity media. They reported that the adversarial objective alone does not guarantee that the reconstructed saturation field is consistent with fluid conservation, and that augmenting the generator loss with a physically motivated regularization term improves this consistency. Their work demonstrates that the adversarial framework can be coupled with conservation-based regularization at the level of a single saturation channel.

Aslam, Li, and Yan [47] considered the related problem of high-contrast channelized media. They proposed a hybrid multiscale framework that combines a pixel-wise error with auxiliary terms penalizing prediction error at well locations, deviations of spatial gradients, total spatial variation, and the local diffusion residual. The pipeline processes dynamic flow information alongside static permeability maps. They reported that this composite objective improves the recovery of pressure and saturation fields and reduces the inference cost compared to fine-scale numerical simulation. Their study shows that combining a regression objective with physics-aware regularizers can recover sharp flow boundaries while remaining computationally tractable.

Despite this progress in deep super-resolution for single-porosity reservoirs, a clear gap remains for naturally fractured formations. To the best of our knowledge, no published study has addressed the simultaneous spatial reconstruction of the three coupled state variables that characterize the dual-porosity formulation, namely global pressure, fracture saturation, and matrix block saturation. Existing physics-informed adversarial frameworks, including those of Doloi et al. [46] and Aslam et al. [47], target single-continuum models and rely on conservation criteria adapted to that setting.

2.6 Conclusions of the Literature Review

The literature surveyed in this chapter covers four interconnected areas. The dual-porosity formulation, examined in Section 2.1, describes the multiphase flow through a coupled system of equations in three state variables and is known to develop sharp displacement fronts that coarse-grid numerical schemes resolve poorly. The discussion of numerical modeling in Section 2.2 formalizes the trade-off between truncation error and computational cost that characterizes any finite-difference solver of this system. The survey of evaluation metrics in Section 2.3 establishes that point-wise, structural, radiometric, gradient-based, and physics-aware criteria each capture a distinct aspect of a reconstruction. The discussion of super-resolution architectures in Sections 2.4 and 2.5 traces the evolution from convolutional regression backbones to adversarial generators and to architectures augmented with explicit physical regularization.

One gap is visible across these four areas. Deep super-resolution methods for reservoir flow have been developed and validated on single-continuum models, but the simultaneous reconstruction of the three coupled state variables of a dual-porosity formulation has not been addressed in the published literature.

Chapter 3

Methodology

This chapter formalizes the reconstruction task addressed in the thesis, defines the experimental contrasts that test the hypotheses stated in the introduction, and specifies the architectural, training, and evaluation components used throughout the remaining chapters.

3.1 Problem Formulation

Reservoir simulation produces a sequence of spatial snapshots that describe the hydrodynamic state of a fractured-porous medium at discrete instants of time. Each snapshot is a three-channel field comprising the global pressure P, the water saturation in the fracture network S, and the water saturation in the matrix blocks $\bar{S}$. The reconstruction task is formulated at the level of a single time step: for any fixed instant of the simulation, the network receives a coarse-grid realization of these three channels and produces a fine-grid estimate of the same three channels. The temporal evolution is obtained by applying the same reconstruction independently at each time step.

Let $X^{LR} \in \mathbb{R}^{C\times h\times w}$ denote the coarse-grid low-resolution (LR) snapshot with C channels and spatial dimensions $h \times w$, where the channels are the three coupled state fields introduced above, and let $X^{HR} \in \mathbb{R}^{C\times H\times W}$ denote the corresponding fine-grid high-resolution (HR) snapshot with $H = s \cdot h$ and $W = s \cdot w$, where s is the upscaling factor applied identically along both spatial axes. A super-resolution generator is a parametric mapping

$$G_\theta : X^{LR} \mapsto \widehat{X}^{HR} \in \mathbb{R}^{C\times H\times W}, \tag{3.1}$$

parameterized by θ. Training consists in fitting θ to a finite collection of paired snapshots $\mathcal{D} = \{(X_n^{LR}, X_n^{HR})\}_n$ produced by the numerical simulator under varied physical and operational parameters.

The formulation rests on three assumptions. First, the spatial domain is restricted to a two-dimensional axisymmetric radial-vertical section of the reservoir, consistent with the geometry of the simulator used to generate the dataset. Second, the upscaling factor s is fixed across all experiments and is applied identically to both the radial and the vertical resolution of the computational grid. Third, the coarse-grid input is an independent numerical solution of the governing equations on a coarse mesh rather than a downsampled version of the fine-grid target, so the mapping that G_θ has to fit recovers physical information lost during coarse-grid simulation rather than inverting a known downsampling operator.

The two hypotheses formulated in the introduction translate into two experimental contrasts that share a common evaluation protocol. The first contrast compares residual generators of different depth, trained with the same pixel-wise objective, on the point-wise reconstruction error of the three-channel field. The second contrast compares the same generators before and after fine-tuning

within an adversarial framework, with the comparison performed jointly across point-wise metrics and across structural and spectral metrics, so that the trade-off between the two metric groups is measured.

3.2 Dataset Construction

3.2.1 Numerical Simulator

The training corpus is produced by a numerical solver of the dual-porosity formulation of Section 2.1 on a two-dimensional axisymmetric domain. The spatial coordinates are the radial distance r from the well axis and the vertical coordinate z. The computational domain is bounded internally by the wellbore radius R_{well} and externally by the drainage radius V_L, with a structurally fixed partition into K horizontal layers indexed by $k = 1, \ldots, K$. Each layer k carries its own thickness h_k, vertical discretization $n_z^{(k)}$, fracture and matrix porosities, irreducible and maximum water saturations, and absolute permeabilities in fracture and matrix sub-systems. The radial grid is uniform with step $\Delta r = V_L/N_X$, the vertical step inside each layer is uniform and equal to $h_k/n_z^{(k)}$, and the vertical grid is the concatenation of the layer-wise vertical partitions, so the total number of vertical cells is $N_Z = \sum_k n_z^{(k)}$.

The solver advances the simulation in time with a fixed time step Δt using a sequential implicit-explicit scheme. At every step the pressure equation (2.3) is linearized and solved implicitly by an iterative procedure that alternates a block-tridiagonal radial sweep with a global update of the boundary fluxes, until the combined update norm of the pressure field falls below a tolerance ε_p. Once the pressure field is updated, the saturation equations (2.4) and (2.5) are advanced

explicitly through N_{dr} drainage sub-steps of size $\Delta t/N_{\mathrm{dr}}$. The inter-porosity exchange term q_Σ of Equation (2.1) enters both saturation equations and is evaluated at every sub-step from the current saturation profile and the relative phase mobilities; the apparent viscosity of the oil phase depends on the local Darcy velocity through a rheological closure parameterized by the constants X_A and X_D. Relative permeabilities follow a power law in the matrix sub-system, with empirical exponents 3.13 for the wetting phase and 2.73 for the non-wetting phase, and a linear law in the fracture sub-system.

The boundary conditions match the operational setting of a producing well in a finite drainage area with constant-pressure outer support. On the inner boundary $r = R_{\mathrm{well}}$ the well flow rate Q_{well} is prescribed, which fixes the radial Darcy flux entering the wellbore. On the outer boundary $r = V_L$ the pressure is held at the constant value P_{c}. The initial pressure field is a steady-state radial profile consistent with the prescribed boundary conditions, and the initial saturation distribution places the hydrocarbon-bearing layers at their irreducible water saturation and the underlying aquifer layer at full water saturation.

For every simulation case the solver is executed twice in parallel with identical physical parameters and identical boundary conditions but on two different computational grids. The fine-grid (HR) run uses N_X^{HR} radial cells and a vertical discretization $s \cdot n_z^{(k)}$ in each layer, where s is the spatial upscaling factor of the problem formulation. The coarse-grid (LR) run uses $N_X^{LR} = N_X^{HR}/s$ radial cells and the layer-wise vertical discretization $n_z^{(k)}$ prescribed by the simulation case. The two runs are therefore independent numerical solutions of the same dual-porosity system, and the LR snapshot is not a downsampled version of the HR snapshot but a separate realization that contains its own numerical diffusion and lacks subgrid detail. At inference time only the coarse-grid solution is available,

and the super-resolution generator maps it to the corresponding fine-grid solution.

A simulation case is characterized by the full vector of physical and operational parameters of the dual-porosity formulation, by the well control regime, and by the layer-wise petrophysical properties. At the end of each time step the solver records the three coupled state fields P, S and $\bar{S}$ on both grids, together with the integral quantities recorded at the same instant: the bottomhole pressure, the cumulative fluid production, the water cut, the oil recovery factors of the two sub-systems, and the mass-balance error indicators. The recorded snapshots and the corresponding scalar metadata are stored in a single archive per case.

3.2.2 Parameter Space and Sampling Strategy

A training corpus that supports the experimental contrasts of the problem formulation must cover a wide range of operating regimes and petrophysical configurations. To this end, the dataset is generated as a campaign of simulation cases in which a subset of the parameters is randomized over a prescribed domain while the remaining parameters are held fixed at reference values typical of a fractured-porous reservoir. The randomized subset contains D parameters and falls into three groups. The first group describes the well control regime and the near-wellbore zone and includes the well flow rate Q_{well}, the contour pressure P_{c}, the wellbore radius R_{well}, the effective near-wellbore viscosity μ_{nw}, and the parameters X_A, X_D of the matrix mobility closure. The second group describes the numerical configuration of the solver and includes the number of drainage substeps N_{dr} and the saturation cutoff thresholds introduced in the sampling strategy. The third group consists of layer-wise petrophysical properties, primarily the absolute permeabilities of fracture and matrix in each layer and small shifts of the

irreducible and maximum water saturations applied identically across the layers.

The parameter domain is sampled with Latin Hypercube Sampling [48] (LHS), which generates a stratified-randomized design of N cases over the D-dimensional unit cube and maps each marginal coordinate to the corresponding physical range. Parameters with a wide dynamic range, such as the absolute permeabilities and the saturation cutoff thresholds, are sampled on a logarithmic scale; the remaining parameters are sampled linearly. LHS is preferred to plain uniform random sampling because the projection of its design onto every individual parameter axis is a stratified sample of size N, which improves marginal coverage at the same total budget.

Every sampled case is subjected to a physical admissibility filter before being submitted to the solver. The filter rejects parameter combinations that violate basic constraints of the formulation, including positivity of the time step, positivity and ordering of the irreducible and maximum saturations within every layer, sufficient simulated time horizon to cover the requested number of snapshots, and compatibility between the prescribed well flow rate Q_{well} and the radial geometry. Cases that pass the filter are dispatched to the solver, and their archives are added to the corpus.

The resulting dataset is partitioned into a training subset, a validation subset, and a test subset at the level of complete simulation archives, with proportions of 0.70, 0.15, and 0.15 respectively. The split is performed by a deterministic shuffle with a fixed seed, so the partition is exactly reproducible. Partitioning at the archive level is adopted because snapshots from the same simulation separated by a small number of time steps are strongly correlated in space; placing such snapshots into different subsets would induce a strong correlation between training and validation samples and inflate the apparent generalization performance of the

model.

3.3 Preprocessing

The three state channels of the snapshot have different physical ranges. The global pressure P takes values on the scale of several tens of atmospheres and has no natural upper bound: its maximum depends on the operational regime and on the petrophysical configuration of the case. The water saturations S and $\bar{S}$, in contrast, are bounded by construction in the physically admissible saturation range and follow distributions concentrated near the two endpoints of this range. The normalization strategy is selected per channel so that each input quantity is presented to the network on a comparable numerical scale while its physical interpretation is preserved.

The pressure channel is standardized by subtracting the per-channel mean and dividing by the per-channel standard deviation, which produces an input with zero mean and unit variance, appropriate for a quantity without natural bounds. The two saturation channels are normalized by an affine min-max transform that maps the per-channel range observed in the training subset to the unit interval $[0, 1]$, which keeps the values within a compact range and preserves their physical meaning as bounded saturations.

The shift and scale parameters of both transforms are estimated once from the training subset of the dataset and are stored as a single set of constants reused for all subsequent operations. Validation, test, and inference samples are normalized with the same constants estimated on the training subset, so that no information about the held-out subsets leaks into the normalization step. Low-resolution and high-resolution snapshots are normalized with separate sets of

constants estimated on their respective marginal distributions, because numerical diffusion on the coarse grid produces a different distribution of values than the fine-grid reference even for identical physical parameters. The normalization is invertible, and its inverse is applied to the network's output before computing physically interpretable error metrics.

3.4 Generator Architectures

The first experimental contrast requires three super-resolution generators of different depth and parameter count that share the same training regime, so that the effect of capacity on the point-wise reconstruction error can be isolated. The three generators differ only in the body that fills the shared skeleton described next; their building blocks are taken from the residual super-resolution literature reviewed in Section 2.4.1.

Common Design

All three generators map the coarse-grid snapshot X^{LR} to the fine-grid estimate $\widehat{X}^{HR}$ through a common pipeline of five stages, shown in Figure 3.1.

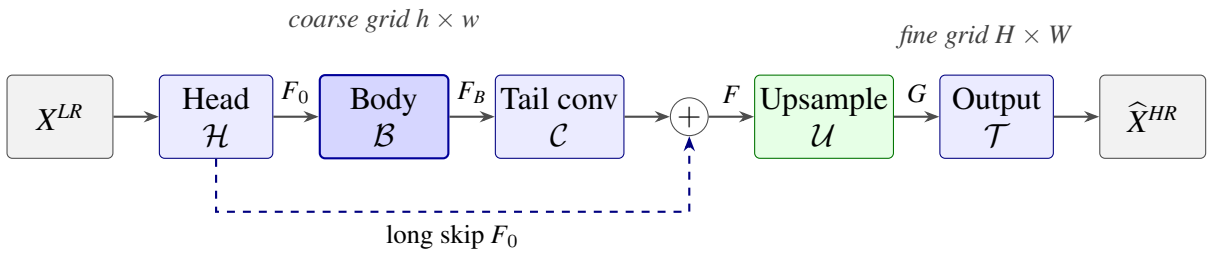

Fig. 3.1. Common generator skeleton.

The head $\mathcal{H}$ is a single 3×3 convolution that lifts the three input channels into a feature tensor F_0 of fixed width C_f at the low-resolution spatial size. The

body $\mathcal{B}$ is a stack of repeated building blocks that processes the feature tensor at the same width C_f and at the same low-resolution spatial size. The body tail convolution $\mathcal{C}$ is a single 3×3 convolution applied to the body output. The long skip connection adds F_0 to the output of $\mathcal{C}$, so the body and its tail convolution learn the residual not captured by the head alone. The upsampling tail $\mathcal{U}$ raises the spatial resolution by the factor s of the problem formulation and keeps the feature width at C_f. The output projection $\mathcal{T}$ is a single 3×3 convolution that maps the upsampled features back into the three output channels of $\widehat{X}^{HR}$. The three generators differ in the composition of the body $\mathcal{B}$, while sharing the same head $\mathcal{H}$, body tail convolution $\mathcal{C}$, long skip connection, upsampling tail $\mathcal{U}$, and output projection $\mathcal{T}$.

The pipeline of Figure 3.1 realizes the post-upsampling pattern introduced by Shi et al. [30] and discussed in Section 2.4.1. All convolutions of the head, the body, and the body tail operate on feature tensors at the coarse-grid spatial size $h \times w$. Only the upsampling tail $\mathcal{U}$ and the output projection $\mathcal{T}$ act on tensors at the fine-grid spatial size $H \times W = (s \cdot h) \times (s \cdot w)$. This placement keeps the most expensive convolutions on the smaller feature maps and concentrates the resolution increase in the sub-pixel convolution layers of the tail.

The upsampling tail is implemented through sub-pixel convolution rather than through a fixed interpolation kernel or through transposed convolution. A sub-pixel convolution stage that increases the spatial resolution by a factor of two consists of a 3×3 convolution that maps a feature tensor of width C_f at low resolution into a tensor of width $4C_f$ at the same low resolution, followed by a pixel-shuffle operator that rearranges these $4C_f$ channels into C_f channels at twice the spatial resolution. The pixel-shuffle operator is a deterministic permutation of tensor elements with no trainable parameters; the learnable part of the upsampling

stage is concentrated in the preceding convolution, which can therefore learn the high-frequency content of the upsampled output. For an upsampling factor $s = 4$ the tail applies two such stages in sequence. The information that the network has to recover is high-frequency by construction: the displacement fronts and the fine-scale heterogeneities that the coarse-grid simulation does not resolve carry energy at spatial frequencies that the coarse grid cannot represent, and any fixed interpolation kernel is a low-pass filter that suppresses this high-frequency content. Sub-pixel convolution learns the upsampling operator from data rather than fixing a predetermined frequency response, and avoids the checkerboard artifacts of transposed convolution at strides greater than one.

None of the three generators uses Batch Normalization. Batch normalization removes the per-sample intensity and contrast information present in the feature maps, which conflicts with the requirements of super-resolution reconstruction, as discussed for the EDSR family by Lim et al. [34] in Section 2.4.1. The three bodies described in the remainder of this section fill the body stage $\mathcal{B}$ of Figure 3.1 with progressively different trade-offs between depth and parameter count.

3.4.1 Modified Super-Resolution Residual Network

The Modified Super-Resolution Residual Network (MSRResNet) generator fills the body stage of the common skeleton with a stack of identical residual blocks. The generator follows the SRResNet design of Ledig et al. [33], shown in Figure 3.2, with the Batch Normalization layers of the original removed for the reason stated in the common design.

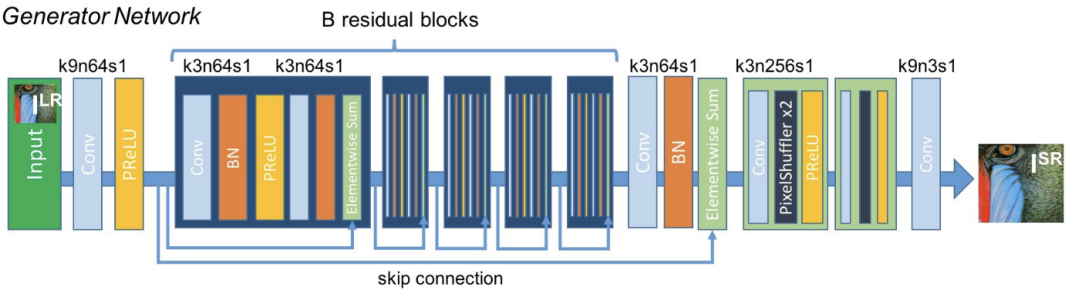

Fig. 3.2. SRResNet generator, after Ledig et al. [33].

Each residual block has the structure

$$x_{i+1} = x_i + \lambda \cdot \mathrm{Conv}_{3\times3}\big(\mathrm{ReLU}\big(\mathrm{Conv}_{3\times3}(x_i)\big)\big), \tag{3.2}$$

where both convolutions operate at the body width C_f and $\lambda \in (0, 1)$ is a constant residual-scaling factor that stabilizes the optimization of the residual stack. The body of the generator is the composition of N_B^{MSR} such blocks. The upsampling tail follows the common skeleton but, in contrast to the other two generators, inserts a ReLU non-linearity after each pixel-shuffle operator. With the smallest block count of the three bodies, MSRResNet is the shallow end of the depth-versus-capacity comparison of the first experimental contrast.

3.4.2 Enhanced Deep Super-Resolution

The Enhanced Deep Super-Resolution (EDSR) generator of Lim et al. [34], shown in Figure 3.3, fills the body stage with the same residual block as MSRResNet, defined by Equation (3.2), at the same body width C_f and residual-scaling factor λ.

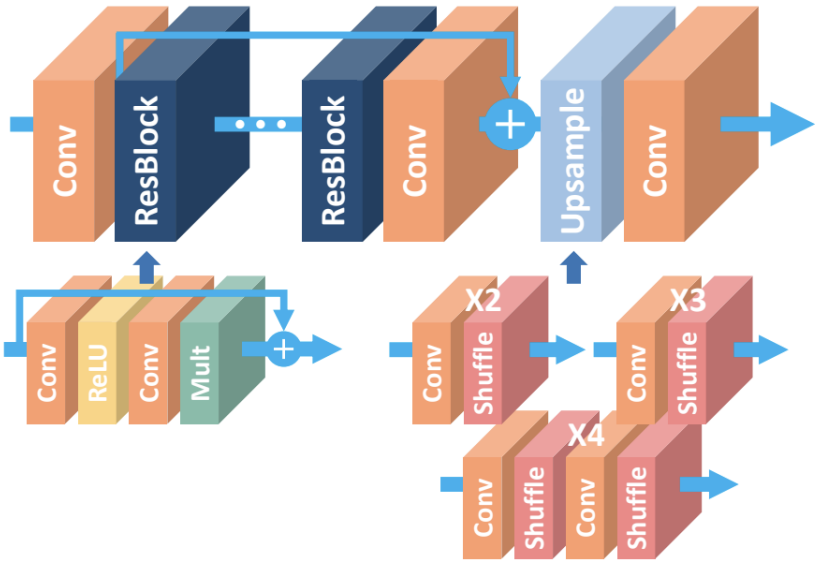

Fig. 3.3. EDSR generator, after Lim et al. [34].

EDSR and MSRResNet thus use the same residual block without Batch Normalization, the EDSR form of which is shown in Figure 3.4, and differ only in body depth: the EDSR body depth N_B^{EDSR} is chosen larger than N_B^{MSR}.

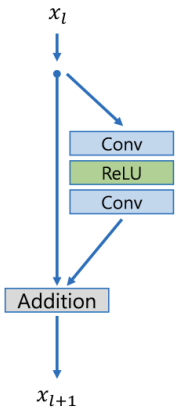

Fig. 3.4. EDSR residual block, after Lim et al. [34].

The upsampling tail uses sub-pixel convolution as in the common skeleton but does not apply a non-linearity after the pixel-shuffle operators. Trained on the coarse-grid input in the same regime as MSRResNet but with a deeper body,

EDSR is the deep end of the depth-versus-capacity comparison.

3.4.3 Residual Feature Distillation Network

The Residual Feature Distillation Network (RFDN) of Liu et al. [36], introduced in Section 2.4.1 and shown in Figure 3.5, fills the body stage with residual feature distillation blocks (RFDBs) at a smaller body width $C_f^{\mathrm{RFDN}} < C_f$ than MSRResNet and EDSR. Rather than reaching capacity through depth, RFDN reaches it through the richer internal structure of its block.

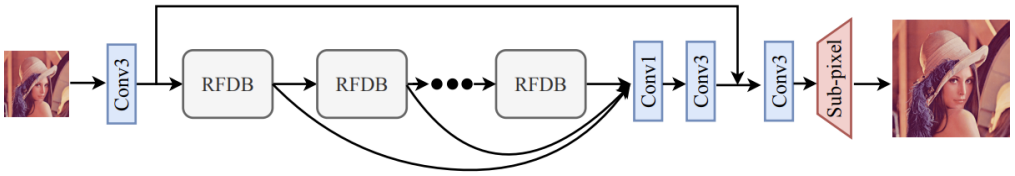

Fig. 3.5. RFDN architecture, after Liu et al. [36].

The block has two interleaved paths, shown in Figure 3.6. Let x denote the input to the block, with channel dimension C_f^{RFDN}. The refinement path produces three intermediate tensors r_1, r_2, r_3 at the same channel dimension as x through the recurrence

$$r_k = \mathrm{LeakyReLU}\big(\mathrm{Conv}_{3\times3}(r_{k-1}) + r_{k-1}\big), \quad k = 1,2,3, \quad r_0 = x, \tag{3.3}$$

and the distillation path produces four tensors d_1, d_2, d_3, d_4 at a reduced channel dimension $C_d = C_f^{\mathrm{RFDN}}/2$ through

$$\begin{aligned} d_k &= \mathrm{LeakyReLU}\big(\mathrm{Conv}_{1\times1}(r_{k-1})\big), \quad k = 1,2,3, \\ d_4 &= \mathrm{LeakyReLU}\big(\mathrm{Conv}_{3\times3}(r_3)\big). \end{aligned} \tag{3.4}$$

The first three distillation tensors are compact 1×1 projections of the successive refinement tensors, and the last is a 3×3 projection of the deepest.

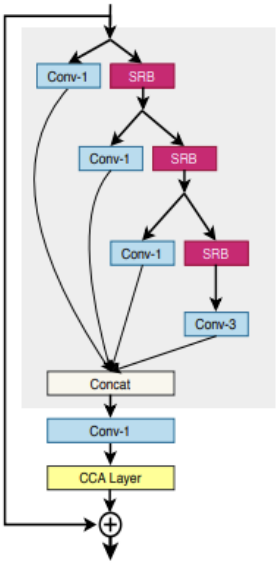

Fig. 3.6. RFDB, after Liu et al. [36].

The four distillation tensors are concatenated along the channel axis and fused by a 1×1 convolution into a tensor of the original channel dimension C_f^{RFDN}, which is then passed through the enhanced spatial attention sub-network of Figure 3.6 that computes a per-pixel attention map and re-weights the input features. The output of the block is the sum of the attention-weighted tensor and the block input x. The outputs of the N_B^{RFDN} body blocks are concatenated along the channel axis and reduced by a 1×1 convolution back to the body width C_f^{RFDN}. The resulting tensor is the body output F_B of the common skeleton, after which the body tail convolution, the long skip connection, the upsampling tail, and the output projection of Figure 3.1 are inherited unchanged; the upsampling tail uses sub-pixel convolution without a non-linearity after the pixel-shuffle operators. With fewer body blocks of richer internal structure and the lowest parameter count of the three, RFDN contributes a third combination of depth and parameter

count to the first experimental contrast.

3.5 Discriminator

The second experimental contrast compares the three generators before and after fine-tuning within an adversarial framework. The adversarial training requires a discriminator that complements the pixel-wise objective with a learning signal sensitive to the high-frequency structure of the reconstruction. This discriminator is a patch-level critic, in the spirit of those used in adversarial super-resolution [33]; it is used only during the adversarial training of the generators and is discarded afterwards, so it is not part of the inference pipeline of the surrogate model. Its structure, from the residual input to the map of patch logits, is shown in Figure 3.7.

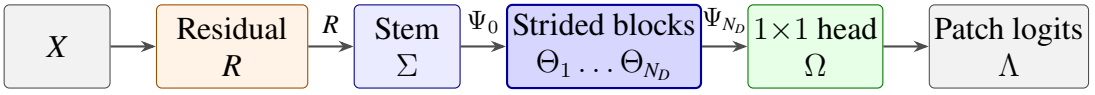

Fig. 3.7. Patch discriminator.

Rather than emitting a single scalar that classifies the whole snapshot as real or fake, the discriminator emits a two-dimensional map of patch logits Λ, each output position classifying its own local region of the input. The effective receptive field of every position covers one patch of the snapshot, so each logit evaluates the local high-frequency structure of that patch and the adversarial loss is the average of the per-patch terms. A discriminator that instead returned a single score for the whole snapshot would tend to bias the generator towards over-smoothed reconstructions, because its gradient rewards low-frequency consistency more strongly than local sharpness.

The input to the critic is not the candidate snapshot X itself but the residual

$$R(X, X^{LR}) = X - \mathrm{Bicubic}_s(X^{LR}), \tag{3.5}$$

where $\mathrm{Bicubic}_s(\cdot)$ denotes bicubic interpolation by the upscaling factor s of the problem formulation. Bicubic upsampling of X^{LR} produces a parameter-free, low-pass reference of the same spatial resolution as X, so the subtraction removes the low-frequency content already present in that reference and concentrates the input on the high-frequency residual that distinguishes a faithful reconstruction from an over-smoothed or hallucinated one. The same transformation is applied to the real samples $X = X^{HR}$ and to the fake samples $X = \widehat{X}^{HR}$, so the network compares the residual signals of the two distributions directly.

The residual is processed by the three stages of Figure 3.7. The stem Σ is a single 3×3 convolution with stride one that maps the three channels of the residual into a feature tensor Ψ_0 of base width b, followed by a leaky rectified linear non-linearity. The body is a stack of N_D strided blocks $\Theta_1, \ldots, \Theta_{N_D}$; each block applies a 3×3 convolution with stride two that halves the spatial dimensions and doubles the channel dimension up to a cap $C_{\max}$, a leaky rectified linear non-linearity, a 3×3 convolution with stride one that preserves the channel dimension, and a second leaky rectified linear non-linearity. Across the body the spatial dimensions are reduced by a factor of 2^{N_D} while the channel dimension grows to $C_{\max}$. The head Ω is a single 1×1 convolution without non-linearity that projects every spatial position of the final tensor Ψ_{N_D} to a single logit, producing the patch-logit map Λ.

Every convolutional layer of the discriminator is constrained by spectral normalization [40], in the form introduced in Section 2.4.2: each weight W is

replaced by $W/\sigma(W)$, where $\sigma(W)$ estimates the largest singular value of the corresponding linear operator. The Lipschitz constant of each layer is then at most one, which controls the Lipschitz constant of the discriminator as a whole, stabilizes the adversarial training, and prevents it from dominating the generator early in the optimization. Because the discriminator operates on the residual rather than on the raw snapshot and scores every patch independently, its gradient drives the generator towards locally sharp, high-frequency detail rather than towards global low-frequency agreement, which is the signal that the pixel-wise objective alone cannot supply.

3.6 Loss Functions

The generators are trained against two distinct objectives, depending on the experimental contrast in which they participate. A pixel-wise regression objective fits the generators to the high-resolution targets and underlies the first experimental contrast. A composite objective adds a domain-specific perceptual term, an optional reservoir-physics regularization, and an adversarial term computed from the discriminator; it is used in the adversarial fine-tuning that underlies the second experimental contrast.

3.6.1 Regression Objective

The pixel-wise discrepancy between the reconstruction $\widehat{X}^{HR}$ and the target X^{HR} is measured by the L_1 norm. Following Mathieu et al. [22] and the analysis of point-wise metrics in Section 2.3, the L_1 norm is preferred over the L_2 norm because it is more robust to the sharp transitions at the displacement fronts of

the saturation channels and yields less smoothed reconstructions. The regression objective is therefore

$$\mathcal{L}_{\mathrm{px}}\big(\widehat{X}^{HR}, X^{HR}\big) = \frac{1}{C\,H\,W}\,\big\|\widehat{X}^{HR} - X^{HR}\big\|_1, \tag{3.6}$$

where C, H, W are the channel and spatial dimensions of the reconstruction and the entry-wise L_1 norm of a tensor A is

$$\|A\|_1 = \sum_i \mid a_i \mid, \tag{3.7}$$

the sum running over all entries a_i of A.

The three generators are trained first with the regression objective and then fine-tuned with the composite objective defined below.

3.6.2 Composite Objective

The composite objective combines the pixel-wise term of Equation (3.6) with a domain-specific perceptual term, an optional reservoir-physics regularization, and an adversarial term. The generator minimizes

$$\mathcal{L}_G = \lambda_{\mathrm{px}}\,\mathcal{L}_{\mathrm{px}} + \lambda_{\mathrm{perc}}\,\mathcal{L}_{\mathrm{perc}} + \lambda_{\mathrm{phys}}\,\mathcal{L}_{\mathrm{phys}} + \lambda_{\mathrm{adv}}\,\mathcal{L}_{G,\mathrm{adv}}, \tag{3.8}$$

where the four non-negative weights $\lambda_{\mathrm{px}}, \lambda_{\mathrm{perc}}, \lambda_{\mathrm{phys}}, \lambda_{\mathrm{adv}}$ control the relative contribution of the four components. The discriminator is trained jointly on its own objective $\mathcal{L}_D$, defined below.

The first additional component is a domain-specific perceptual term. The classical perceptual loss introduced by Ledig et al. [33] and discussed in Sec-

tion 2.4.2 compares pre-trained convolutional features of the reconstruction and the target in a feature space learned on natural images. Pressure and saturation fields are not natural images, and features extracted by a network trained on photographic data have no physical interpretation here. The perceptual term is therefore replaced by a domain-specific objective on the spatial gradients and the amplitude spectra of the reconstruction and the target. The gradient component is defined through the Sobel operators [27] $\mathbf{S}_x, \mathbf{S}_y$ as

$$\begin{aligned}\mathcal{L}_{\text{grad}} = \tfrac{1}{2}\Big(&\left\|\mathbf{S}_x * \widehat{X}^{HR} - \mathbf{S}_x * X^{HR}\right\|_1 \\ &+ \left\|\mathbf{S}_y * \widehat{X}^{HR} - \mathbf{S}_y * X^{HR}\right\|_1\Big),\end{aligned} \tag{3.9}$$

where $*$ denotes channel-wise convolution. The spectral component compares the amplitude spectra of the reconstruction and the target through the unitary two-dimensional discrete Fourier transform $\mathcal{F}$:

$$\mathcal{L}_{\text{spec}} = \left\| \,|\, \mathcal{F}(\widehat{X}^{HR}) \,|\, - \,|\, \mathcal{F}(X^{HR}) \,|\, \right\|_1, \tag{3.10}$$

where $|\cdot|$ denotes the entry-wise magnitude. The full domain-specific perceptual term is the weighted sum

$$\mathcal{L}_{\text{perc}} = w_g\, \mathcal{L}_{\text{grad}} + w_s\, \mathcal{L}_{\text{spec}}. \tag{3.11}$$

The gradient component penalizes misalignment of the displacement fronts, and the spectral component penalizes mismatch of the energy distribution across spatial frequencies, including the high frequencies that the pixel-wise term weakly constrains.

The second additional component is a regularization term that injects two

priors from the dual-porosity formulation of Section 2.1: spatial smoothness of the pressure field, and consistency of the saturation fronts. It is the weighted sum

$$\mathcal{L}_{\text{phys}} = w_p \, \mathcal{L}_{\text{smooth}}(\widehat{P}) + w_e \, \mathcal{L}_{\text{edge}}\big(\widehat{S}, \widehat{\bar{S}}, S, \bar{S}\big). \tag{3.12}$$

The smoothness penalty applies a 5-point discrete Laplacian Δ_h to the reconstructed pressure channel and penalizes its absolute value:

$$\mathcal{L}_{\text{smooth}}(\widehat{P}) = \big\| \Delta_h \widehat{P} \big\|_1. \tag{3.13}$$

The penalty reflects the parabolic character of the pressure equation (2.3), whose solutions are smooth away from the well, and discourages spurious high-frequency oscillations in the reconstructed pressure field. The edge-consistency penalty compares the magnitudes of the Sobel gradients of the reconstructed and target saturation channels:

$$\mathcal{L}_{\text{edge}} = \tfrac{1}{2} \sum_{c \in \{S, \bar{S}\}} \big\| \, | \, \nabla \widehat{c} \, | - | \, \nabla c \, | \, \big\|_1, \tag{3.14}$$

where $| \, \nabla c \, |$ denotes the magnitude of the Sobel gradient of saturation channel c, the sum running over the two saturation channels S and $\bar{S}$ while the pressure channel P is handled by the smoothness term above. Comparing the magnitudes rather than the gradient vectors themselves preserves the location and the sharpness of the saturation fronts while tolerating small pixel-level misalignments. The regularization term $\mathcal{L}_{\text{phys}}$ is included in the composite objective when $\lambda_{\text{phys}} > 0$.

The third additional component is the adversarial term, computed from the discriminator D in the relativistic average formulation of Jolicoeur-Martineau [39]

discussed in Section 2.4.2. The discriminator estimates whether a real sample is more realistic than the average fake sample, and conversely for fake samples. The relativistic logits are

$$\begin{aligned} \widetilde{D}(X^{HR}, \widehat{X}^{HR}) &= D\big(X^{HR}\big) - \mathbb{E}_{\widehat{X}^{HR}} D\big(\widehat{X}^{HR}\big), \\ \widetilde{D}(\widehat{X}^{HR}, X^{HR}) &= D\big(\widehat{X}^{HR}\big) - \mathbb{E}_{X^{HR}} D\big(X^{HR}\big), \end{aligned} \tag{3.15}$$

where the expectations are estimated by the mean over the current mini-batch. The adversarial objective of the generator is the symmetric binary cross-entropy

$$\begin{aligned} \mathcal{L}_{G,\mathrm{adv}} = \tfrac{1}{2} \Big(& f_{\mathrm{BCE}}\big(\widetilde{D}(X^{HR}, \widehat{X}^{HR}), 0\big) \\ & + f_{\mathrm{BCE}}\big(\widetilde{D}(\widehat{X}^{HR}, X^{HR}), y_r\big) \Big), \end{aligned} \tag{3.16}$$

and the discriminator minimizes the symmetric counterpart

$$\begin{aligned} \mathcal{L}_{D} = \tfrac{1}{2} \Big(& f_{\mathrm{BCE}}\big(\widetilde{D}(X^{HR}, \widehat{X}^{HR}), y_r\big) \\ & + f_{\mathrm{BCE}}\big(\widetilde{D}(\widehat{X}^{HR}, X^{HR}), 0\big) \Big), \end{aligned} \tag{3.17}$$

where $f_{\mathrm{BCE}}(z, y) = -y \log \sigma(z) - (1-y) \log\big(1-\sigma(z)\big)$ is the binary cross-entropy with logits and σ is the logistic function. The target $y_r \in (0, 1)$ implements one-sided label smoothing: training the discriminator against a soft target $y_r < 1$ instead of a hard target of one slows its convergence and stabilizes the adversarial dynamics.

The four weights $\lambda_{\mathrm{px}}, \lambda_{\mathrm{perc}}, \lambda_{\mathrm{phys}}, \lambda_{\mathrm{adv}}$ of Equation (3.8), together with the two pairs (w_g, w_s) and (w_p, w_e) of Equations (3.11) and (3.12), are non-negative scalars that balance the four components against each other. The specific values used in each experiment are stated together with the experiment; they are selected

to preserve the pixel fidelity reached by the regression objective while admitting the additional terms, and to keep the discriminator-generator dynamics stable during training.

3.7 Training Strategy

The two experimental contrasts of this chapter divide the training into two stages. The regression stage trains every generator with the pixel-wise objective $\mathcal{L}_{\mathrm{px}}$ and produces the weights evaluated in the first contrast. The adversarial stage fine-tunes the same three generators with the composite objective $\mathcal{L}_G$ and produces the weights evaluated in the second contrast.

3.7.1 Regression Stage

The regression stage produces three training runs: MSRResNet, EDSR, and RFDN. All three runs minimize the pixel-wise objective $\mathcal{L}_{\mathrm{px}}$ of Equation (3.6) under a single optimization regime and differ only in the generator architecture.

Parameters are updated by AdamW with constant weight decay. The learning rate follows a cosine annealing schedule from an initial value η_0 to a floor $\eta_{\min}$ over the full duration of the stage. Before every update the gradients of each network are rescaled so that their global L_2 norm does not exceed a fixed clip threshold.

An exponential moving average (EMA) of the generator weights, a running average that smooths out the per-step fluctuations of training, is maintained throughout the stage and is updated after every parameter step of the underlying generator by

$$\theta_{t+1}^{\mathrm{EMA}} = \alpha\, \theta_{t}^{\mathrm{EMA}} + (1-\alpha)\, \theta_{t+1}, \tag{3.18}$$

where θ_t are the instantaneous parameters at training step t and $\alpha \in (0, 1)$ is a fixed decay factor. Validation, checkpoint selection, and the construction of the final exported model use the moving-average copy rather than the instantaneous one, because the moving-average parameters are less sensitive to mini-batch noise and produce smoother validation curves.

For all three generators the regression stage produces the initial weights of the adversarial stage.

3.7.2 Adversarial Stage

The adversarial stage fine-tunes each of the three generators under the composite objective $\mathcal{L}_G$ of Equation (3.8) and the discriminator objective $\mathcal{L}_D$ of Equation (3.17). The stage is run once per backbone, so three independent adversarial runs are produced.

The generator is initialized from the best regression-stage checkpoint of the same backbone. The discriminator is initialized from random weights. The two networks are updated alternately on every training batch, the discriminator first, the generator second.

Both networks are optimized with Adam, each with its own optimizer and cosine annealing schedule, from initial learning rates η_G^0 or η_D^0 to the floor $\eta_{\min}$. The discriminator is given a larger initial learning rate than the generator, $\eta_D^0 > \eta_G^0$, following the two time-scale update rule: the generator starts from a regression-stage solution, while the discriminator starts from a random initialization, and the larger learning rate compensates for the longer adaptation path of the discriminator. Before every update the gradients of each network are rescaled so that their global L_2 norm does not exceed a fixed clip threshold, set separately from the

regression stage.

The adversarial stage starts with a two-phase warmup that prepares the generator and the discriminator before the full composite objective is activated. During the first N_D^{w} epochs only the discriminator is updated; the generator parameters are held fixed, and the discriminator alone is trained on the residual input of Equation (3.5), so that it provides a non-trivial classification signal before its gradient enters the generator update. During the next N_G^{w} epochs the generator is updated against the non-adversarial terms of $\mathcal{L}_G$ with the adversarial weight λ_{adv} held at zero, which lets the generator adjust to the gradients of the perceptual and physics terms before the discriminator-induced gradient is added. Once both warmup phases are complete, λ_{adv} is set to its target value and every subsequent update uses the full composite objective. Figure 3.8 summarizes this schedule.

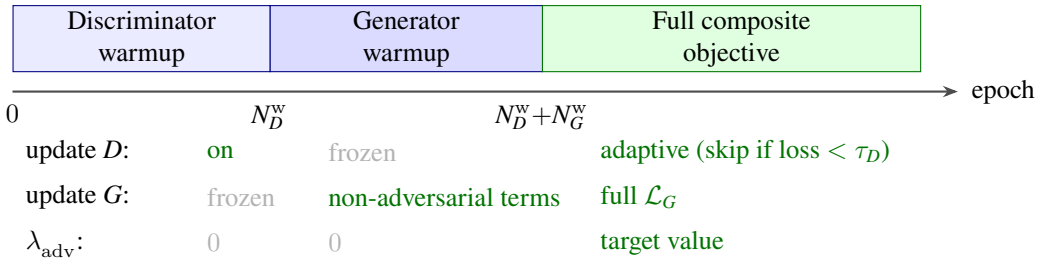

Fig. 3.8. Adversarial training schedule.

Two further mechanisms stabilize the dynamics inside the adversarial phase. The first is an adaptive update rule for the discriminator: when the discriminator loss falls below a threshold τ_D, the discriminator step is skipped on the current batch and only the generator step is performed, so that the discriminator is not optimized further on batches it already classifies well. A periodic refresh overrides this rule at fixed epoch intervals and forces a discriminator update, so that the discriminator parameters keep pace with the generator output as the generator continues to improve. The second mechanism is additive Gaussian noise applied

to the inputs of the discriminator, with the same standard deviation on the real and on the fake samples, and linearly annealed to zero over a prescribed number of epochs. The noise softens the classification problem in the early epochs of the adversarial stage, when the residuals of the generator output and of the high-resolution targets follow different distributions.

The EMA of the generator weights is maintained throughout the adversarial stage in the form of Equation (3.18), with a decay factor that can be set independently of the one used in the regression stage. The discriminator is not averaged. Validation and checkpoint selection use the moving-average copy, as in the regression stage. The moving-average weights at the end of the adversarial stage are the weights of each generator evaluated in the second experimental contrast.

3.8 Evaluation Protocol

All trained generators are evaluated against the fixed suite of metrics formalized in the literature review in Section 2.3, summarized in Table 3.1 in decreasing order of priority. Equation references point to the literature review. PSNR is reported as the mean across the three physical channels P, S, $\bar{S}$; SSIM, GMAE, and ERGAS are global quantities. An experiment satisfies the acceptance criteria if its scores on these four metrics reach their respective acceptance bounds on the held-out test subset. The maximum absolute error (max_ae) and the Spectral Angle Mapper (SAM), the latter expressed in radians, are reported separately as diagnostics: max_ae characterizes the worst-case point deviation and SAM the preservation of inter-channel coupling, and neither enters the acceptance decision. The channel-resolved values of PSNR and MAE for P, S, $\bar{S}$ are also reported, without acceptance bounds, to expose the asymmetry of the reconstruction across the

three channels.

The acceptance bounds of Table 3.1 are tied to the dynamic range of the reconstruction targets defined in the problem formulation. Because the fine-grid fields are smooth almost everywhere outside the displacement fronts, a faithful reconstruction concentrates its point-wise error on a small fraction of the spatial domain, and the bounds on PSNR, GMAE, and ERGAS lie at the level that separates such a reconstruction from one in which the error spreads across the smooth interior. The structural similarity score, in turn, is upper-bounded by the contribution of the sharp transitions at the displacement fronts even when the reconstruction agrees with the target outside them, and the SSIM bound is positioned at the corresponding regime-specific value. A candidate generator is accepted only when its point-wise error stays below the levels appropriate to the physical fields and its local structure around the fronts is preserved.

TABLE 3.1
Evaluation suite.

Metric	Equation	Direction	Acceptance bound
Acceptance metrics			
PSNR	(2.10)	$\uparrow$	≥ 45
SSIM	(2.12)	$\uparrow$	≥ 0.55
GMAE	(2.15)	$\downarrow$	≤ 0.002
ERGAS	(2.13)	$\downarrow$	≤ 0.4
Diagnostic metrics			
max_ae	(2.9)	$\downarrow$	—
SAM	(2.14)	$\downarrow$	—

Taken together, the problem formulation, the dataset construction, the generator and discriminator architectures, the loss functions, the two-stage training strategy, and the evaluation protocol defined in this chapter fully specify the two experimental contrasts. The next chapter describes how these components are

realized in code, how the dataset is generated, and how the trained generators are exported for deployment.

Chapter 4

Implementation

This chapter documents how the methodology of Chapter 3 is realized in code. The material is grouped into four parts: the overall architecture, the data generation campaign, the training and export commands, and the desktop tool that runs the trained generator next to the numerical solver.

4.1 Software Stack Overview

Figure 4.1 shows how the components of the software stack supporting the methodology relate to each other. Two regimes share the same numerical solver. An offline training pipeline produces a trained generator ready for inference, and an online desktop application loads it and applies it either to live coarse-grid input from the solver or to stored archives that include fine-grid references for comparison.

A standalone C# service implements the dual-porosity solver and answers requests through Google Remote Procedure Calls (gRPC), which carries Protocol Buffers messages over the HTTP/2 transport protocol and lets the C# and Python

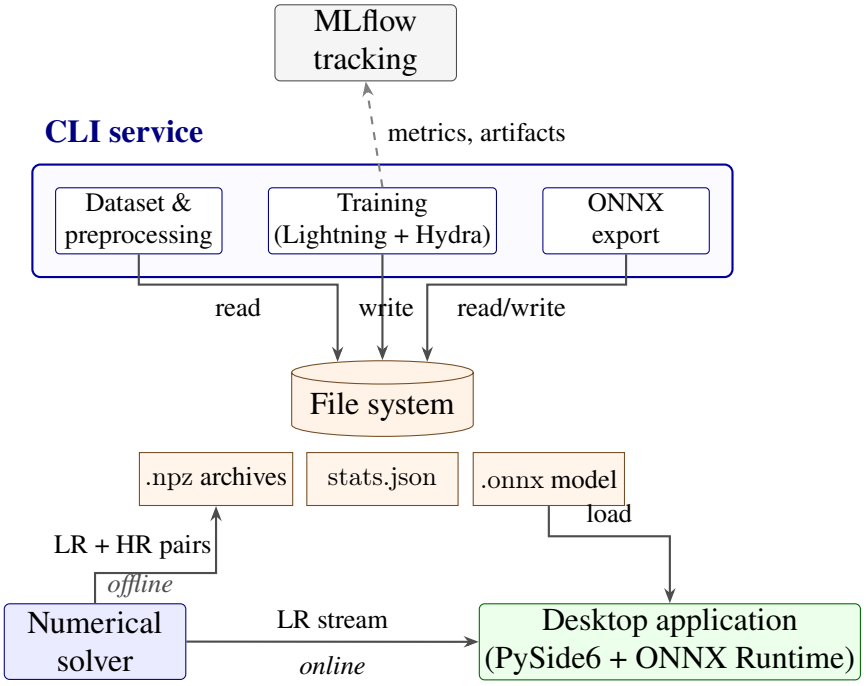

Fig. 4.1. Software architecture of the surrogate-model pipeline.

sides of the project remain independent. During data generation the service runs paired low-resolution and high-resolution simulations on demand and writes each pair, along with the dynamic and static scalar metadata, into a single .npz archive. At runtime it streams the current state of an interactive simulation to the desktop client through the same connection.

Two command-line entry points orchestrate the offline workflow. Both are wired through Hydra, an open-source Python configuration framework developed by Meta that assembles one runtime configuration object from a hierarchy of layered configuration files in the human-readable YAML format and command-line overrides, which makes every experiment reproducible from its stored configuration. The first entry point handles dataset assembly, training, and metric reporting. Training runs under PyTorch Lightning, an opinionated wrapper around PyTorch that pins down the boundary between the network and its loss on one side, and the device placement, checkpoint serialization and validation hooks on the other.

Dataset assembly walks the archive directory, splits the archives deterministically into training, validation, and test subsets, computes per-channel normalization constants from the training split, and writes them to stats.json. The fitting loop consumes the normalized tensors and emits Lightning checkpoints onto disk. MLflow, an open-source platform for tracking machine-learning experiments, attaches to the same loop as an observer and stores scalar metrics, hyperparameter values and intermediate weights of each run; the training code does not depend on MLflow being reachable.

A separate export entry point loads a stored checkpoint together with the normalization constants, rebuilds the generator from the architecture description embedded in the checkpoint, and emits a single file in the Open Neural Network Exchange (ONNX) format. ONNX is a vendor-neutral interchange format for neural-network computation graphs. The export step ends with a side-by-side numerical run of the original PyTorch graph and the exported version to catch divergence introduced by the conversion.

PySide6, the Python bindings for the Qt widget toolkit, hosts the online application. ONNX Runtime, an inference library that executes ONNX graphs on CPU or GPU without pulling in PyTorch as a dependency, handles the forward pass. At startup the application reads the model and the normalization constants from disk and opens a gRPC channel to the solver. The main window splits horizontally into two top-level tabs, Data and Evaluation, that share a common settings dialog for endpoint, paths and rendering preferences.

Three sub-tabs sit inside the Data tab, sharing a left column with a JSON configuration loader and a playback strip (Start, Pause, Step, step batch, interval). On the Runtime sub-tab the user edits the operational controls of an interactive simulation (the radial grid size, the well flow rate, the outer-boundary pressure, the

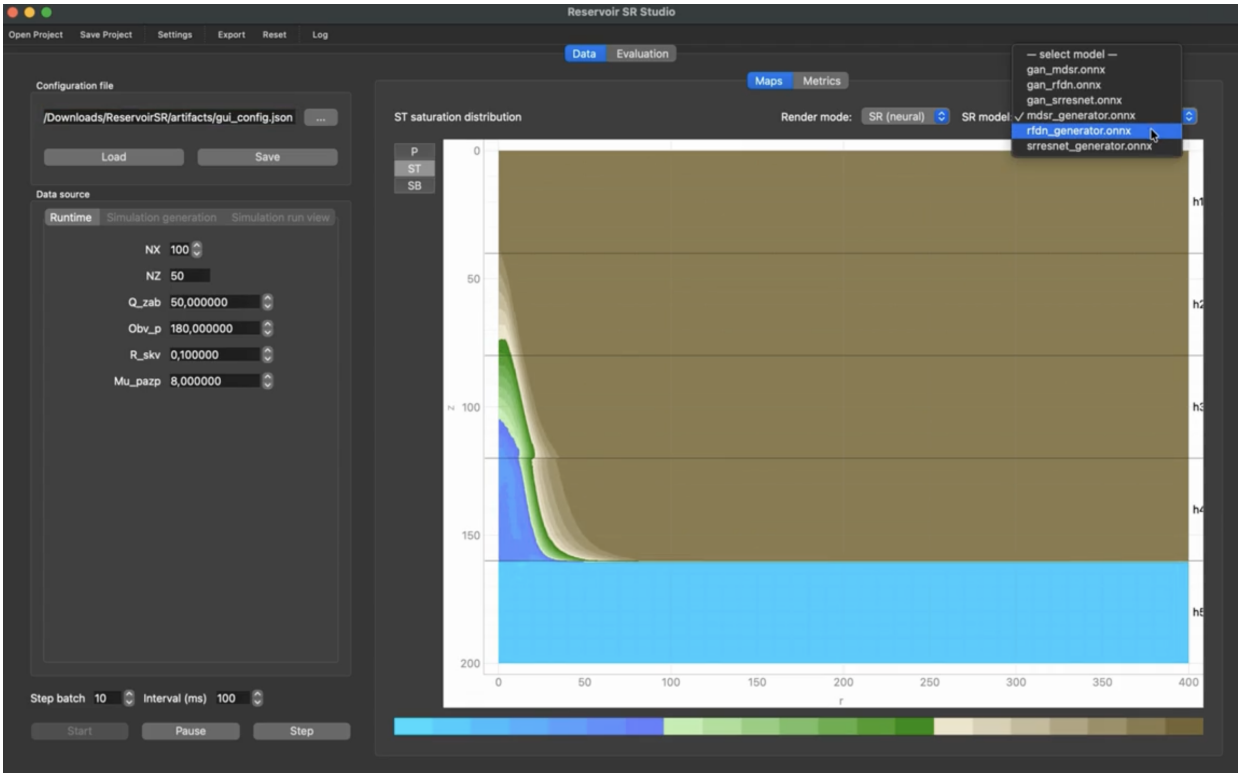

Fig. 4.2. Runtime sub-tab of the Data view with neural super-resolution of the fracture saturation channel.

wellbore radius, and the near-wellbore oil viscosity) and watches the result on the canvas to the right. That canvas combines a Maps view, where one of the three physical channels is selected and the user toggles between linear-interpolation rendering and neural reconstruction backed by an ONNX file picked from a drop-down list, and a Metrics view that plots integral indicators against simulated time. Figure 4.2 captures this sub-tab during a live session, with the fracture saturation channel reconstructed by the trained generator from the coarse-grid solver output.

Dataset jobs go through the Simulation generation sub-tab, whose form collects an output directory, a job identifier, the number of time steps, the snapshot stride, the radial resolutions of the low- and high-resolution runs, the fixed time step, and the pressure-solver tolerance. A mode selector switches between a single job and a Latin Hypercube campaign whose parameters are the sample size, the

random seed, and the worker count, and a progress bar tracks the queue. A third sub-tab, Simulation run view, opens a stored archive or a folder of archives, lists the recorded shapes of the field and scalar tensors, and lets the user step through the timeline at low or high resolution.

On the Evaluation tab the user studies a trained model against a held-out subset of the dataset. Its left column lets the user pick a model file, a split among train, validation and test, and an individual archive inside that split; a slider then walks through the timesteps of the chosen archive, and an optional batch prefetch precomputes the network output for every step in advance to make timeline scrolling smooth. The right column is a three-by-four grid (Figure 4.3): rows correspond to the three physical channels, and columns show, for each channel, the coarse-grid input, the fine-grid reference, the neural reconstruction, and the absolute difference between the reconstruction and the reference, all with per-channel color scales adapted to the local value range.

4.2 Data Generation and Training Configuration

4.2.1 Dataset Generation

Each dataset case is generated by a single job dispatched to the C# component of Section 4.1. The job spins up two independent instances of the dual-porosity solver and advances them in parallel on grids that differ only in their resolution: a coarse 50×100 slice along the radial-vertical axes (LR) and a fine 200×400 counterpart (HR), with the integer scale factor $s = 4$ applied along the radial axis as well as to the per-layer vertical discretization. Both grids share the same five-layer stratigraphy fixed by the solver interface. Three field channels are recorded

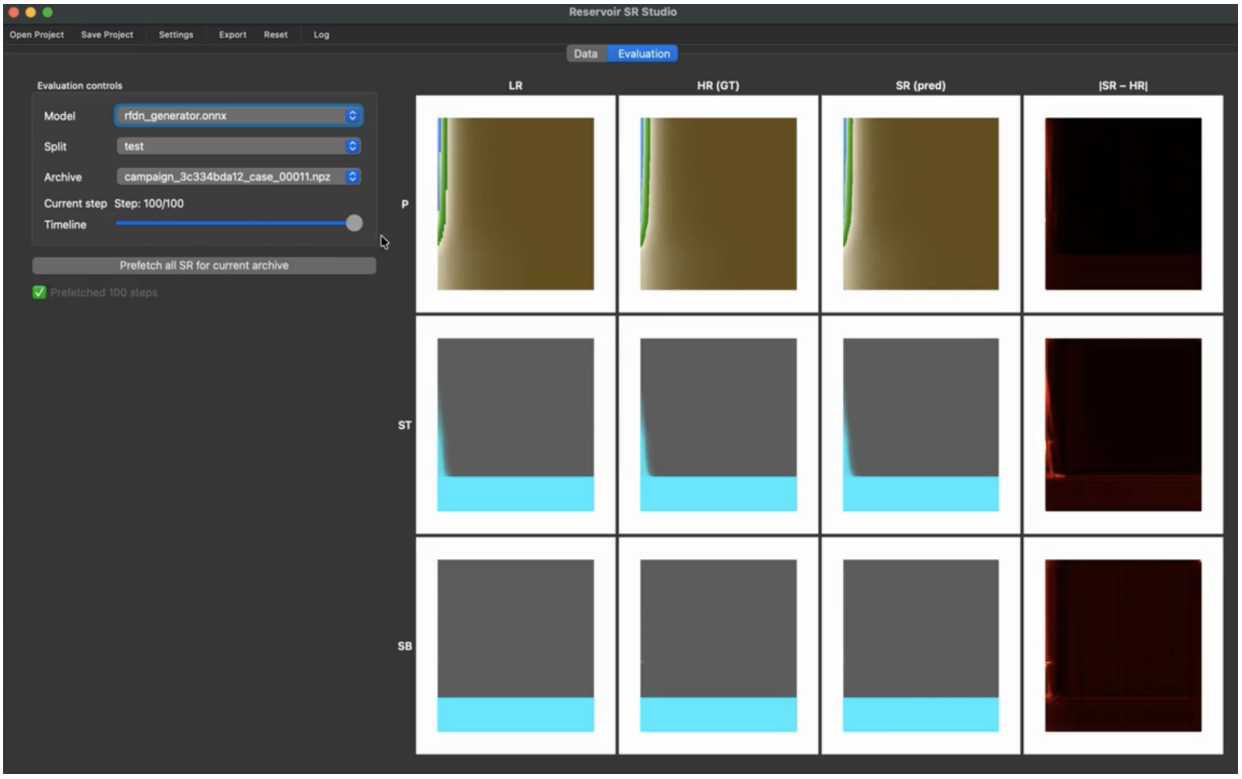

Fig. 4.3. Evaluation tab: three-by-four comparison grid of coarse-grid input, fine-grid reference, neural reconstruction, and absolute difference.

at every snapshot: the global pressure P, the fracture water saturation S, and the matrix water saturation $\bar{S}$.

Time integration is identical on the two grids. Each run advances for 500 steps and writes a snapshot every five of those, which yields $T = 100$ recorded frames per archive. Each frame bundles the three field channels at the corresponding resolution with the dynamic well-response scalars sampled at the same instant, namely the production time, the integral water-cut indicators, the bottom-hole pressure, the field flow rate, the dissipation terms, the temperatures in the matrix and fracture blocks, and the well-block oil rates. The static parameters of the case and the per-layer petrophysical configuration are written once per case. Each container is a single $.\mathrm{npz}$ file that holds the LR and HR field stacks, the per-step dynamic scalars and the static and per-layer scalar metadata, plus a small

JSON manifest with channel names and tensor shapes.

Operational and physical parameters of every case are drawn by Latin Hypercube Sampling (LHS). Their full inventory ($D = 29$ variables grouped into well operating conditions, solver tolerances and per-layer petrophysics) is given in Table 4.1, together with the linear or $\log_{10}$ scale of each range. The drainage sub-step count is the only integer variable. Each draw passes through an admissibility filter inherited from the methodology of Section 3.2.2 before reaching the solver, and rejected cases are discarded. The accepted campaign yields $N = 100$ archives, that is $N \cdot T = 10\,000$ field snapshots.

Discovery and partitioning of those archives happens inside the Lightning data module. It scans the dataset directory for files with the $.\mathrm{npz}$, $.\mathrm{sr}$ and $.\mathrm{zip}$ extensions, shuffles the resulting list with a fixed seed of 42, and assigns the cases to training, validation and test subsets in proportions of 0.70, 0.15 and 0.15. A single pass over the training subset produces per-channel and per-scalar normalization constants, which are saved to $\mathrm{stats.json}$ and reused on every subsequent run. The frame-level loader expands the case-level lists into (archive, time step) pairs and serves each pair as an LR/HR field tensor. Batches of size 128 are produced by 10 parallel workers with pinned host memory; the workers stay alive across epochs, the train loader drops the trailing partial batch, and a least-recently-used (LRU) cache of 128 archives keeps repeated reads off disk.

TABLE 4.1
Sampling space of the generation campaign.

Parameter	Range	Unit	Physical meaning
Well and operational conditions			
Drainage sub-steps	$[8, 20]$	count	explicit saturation sub-steps within one pressure step
Well flow rate	$[15, 70]$	t/day	target well production rate

continued on next page

Table 4.1 – continued from previous page

Parameter	Range	Unit	Physical meaning
Drawdown amplitude	$[120, 480]$	atm	amplitude of well pressure drawdown
Wellbore radius	$[0.05, 0.35]$	m	near-wellbore flow resistance
Contour pressure	$[90, 210]$	atm	pressure at the outer drainage boundary
Shut-down water cut	$[80, 360]$	%	water-cut threshold for well shutdown
Near-wellbore viscosity	$[4.0, 18.0]$	mPa·s	oil viscosity in the near-wellbore zone
Mobility closure, lower	$[0.7, 1.8]$	—	lower bound of the matrix mobility closure
Mobility closure, upper	$[0.05, 0.6]$	—	upper bound of the matrix mobility closure
Gas-nucleus radius	$[3.0, 12.0]$	μm	radius of gas bubble nuclei at the bottomhole
Solver tolerances and time step			
Gas compressibility tol.	$[1.5, 6.0] \times 10^{-3}$	1/MPa	tolerance on the gas compressibility update
Matrix cutoff, lower	$[10^{-5}, 5 \times 10^{-3}]$	—	lower saturation cutoff in matrix blocks
Matrix cutoff, upper	$[10^{-5}, 5 \times 10^{-3}]$	—	upper saturation cutoff in matrix blocks
Fracture cutoff, lower	$[10^{-6}, 10^{-3}]$	—	lower saturation cutoff in fracture network
Fracture cutoff, upper	$[10^{-5}, 5 \times 10^{-3}]$	—	upper saturation cutoff in fracture network
Per-layer petrophysics			
Fracture permeability, high	$[0.05, 0.5]$	Darcy	fracture permeability of high-permeability layers
Matrix permeability, high	$[0.005, 0.05]$	Darcy	matrix permeability of high-permeability layers
Fracture permeability, low	$[0.01, 0.1]$	Darcy	fracture permeability of the low-permeability layer
Matrix permeability, low	$[0.0005, 0.005]$	Darcy	matrix permeability of the low-permeability layer
Fracture S_{wi} shift	$[-0.15, 0.15]$	—	shift of fracture irreducible water saturation
Matrix S_{wi} shift	$[-0.15, 0.15]$	—	shift of matrix-block irreducible water saturation
Fracture $S_{w\max}$ shift	$[-0.15, 0.15]$	—	shift of fracture maximum water saturation
Matrix $S_{w\max}$ shift	$[-0.15, 0.15]$	—	shift of matrix-block maximum water saturation

4.2.2 Training Configuration

A training run is assembled by Hydra from five layered fragments: a globals block with the artifacts directory, the random seed and the logging endpoints; the data block described above; a generator block from the model catalog; a trainer block; and an experiment overlay that names the run and overrides selected values from the lower layers. The catalog carries three regression backbones (a light residual SR network, a deep residual network and a feature-distillation network) and three adversarial bundles that pair each backbone with the patch critic, the perceptual and physics terms, and the relativistic adversarial loss. Ablations reuse the same catalog by switching a single override, for example the pixel-loss module.

Lightning is fixed to one NVIDIA H100 device in mixed-precision bfloat16, with autotuned cuDNN kernels and non-deterministic computation enabled for speed. Train metrics are flushed every twenty optimizer steps; validation fires every ten epochs in the pixel stage and every one or two epochs for the adversarial bundles, where it also triggers the heavy structural metrics; one sample per validation pass is rendered as a four-panel comparison figure. Gradients are clipped by their global L_2 norm at 1.0 in the trainer for regression runs (raised to 5.0 for some of them) and disabled at the trainer level for adversarial runs, where the GAN module performs its own per-network clipping. The regression stage trains each backbone with AdamW under a constant weight decay of 10^{-4}, an initial learning rate between $2 \cdot 10^{-4}$ and $5 \cdot 10^{-4}$ depending on the backbone, and a cosine schedule that anneals to 10^{-5} or 10^{-6} over the full epoch budget (one hundred to one hundred fifty epochs, set per experiment). The default pixel objective is the entry-wise L_1 norm; Charbonnier and squared-error alternatives

are available as interchangeable modules.

For the adversarial fine-tune the best checkpoint of the matching regression run becomes the initial generator and the critic starts from random weights. Both networks are optimized with Adam, with its first- and second-moment decay rates set to $\beta = (0.9, 0.99)$, and a cosine schedule down to 10^{-6}, with the initial rate set to $5 \cdot 10^{-5}$ for the first and $2 \cdot 10^{-4}$ for the second so that the latter compensates for its longer adaptation path. Warmup periods are fixed at five and ten epochs (discriminator-only and then generator-only on the non-adversarial terms); the adversarial weight reaches its target between 0.03 and 0.1 once warmup ends. Input noise of standard deviation 0.02 is injected into both discriminator inputs and annealed linearly to zero over thirty epochs; the critic step is skipped on batches whose last loss fell below 0.35 and is forced again every fifth epoch so that it keeps pace with the improving network; the positive label is smoothed to 0.9. An exponential moving average of the generator weights runs throughout both stages with a decay of 0.999 in the pixel stage and between 0.99 and 0.995 in the adversarial one, and feeds validation, checkpoint selection and the exported artifact.

Run-level instrumentation is shared across every experiment. Scalar metrics, hyperparameters and intermediate weights are streamed to an MLflow server during training; a summary line with parameter counts, validation losses and the headline structural and spectral scores is appended to a single comma-separated values (CSV) report at the end of each run, together with a pass-or-warn status against the acceptance bounds of the evaluation suite. Hydra writes the resolved configuration and the working directory of each launch under a date-stamped path inside the artifacts directory, so every experiment carries its own self-contained record.

4.3 Model Export and Inference

Once training is over, a separate entry point converts the Lightning checkpoint of a chosen run into a portable file. Such a checkpoint stores two parameter copies: the values updated by the last backward pass and the moving-average snapshot kept alongside them. By default the converter takes the moving-average copy when present and falls back to the instantaneous one otherwise, since that copy drives validation and provides the scores reported for the run. The same checkpoint also stores the architecture description of the training session among its saved hyperparameters; this description is applied on top of the export configuration so that the block count, the feature width, and the residual scale are recovered automatically. The generator is then instantiated and the weights are loaded strictly, with any missing or unexpected key aborting the conversion.

Because the desktop client feeds the network with a single tensor of low-resolution channels rather than the dictionary used during training, the generator is wrapped in a thin adapter that injects the tensor into the expected input map. The adapter, in evaluation mode, is handed to the standard PyTorch exporter together with a dummy input of shape $1 \times 3 \times 50 \times 100$ and operator set seventeen; constant folding is enabled, the parameters are baked into the resulting graph, and three axes (batch, height, width) are declared dynamic on both ends so that the same artifact accepts arbitrary batch sizes and spatial dimensions at runtime.

Before the file is released, the exporter compares a fixed-seed Gaussian input pushed through the original PyTorch wrapper and through an ONNX Runtime CPU session: the maximum absolute element-wise difference must stay below 10^{-3}, otherwise the export is rejected. The same kind of session, opened by the desktop client at startup with full graph optimization and four worker threads,

applies the saved per-channel statistics to the incoming coarse-grid tensor, calls the network, and reverses the transform on the output before handing the high-resolution fields to the renderer.

Chapter 5

Evaluation and Discussion

This chapter reports the empirical outcomes of the experimental contrasts formulated in the methodology. The training trajectories of the three pixel-only generators are examined first, with attention to the divergence between the point-wise and the structural quality observed in the second half of the schedule. The adversarial fine-tunes of the same three backbones follow, with the focus on the dynamics of the discriminator-generator pair and on the gain in structural quality the composite objective produces over the pixel-only baseline. A summary closes the chapter, placing the six runs side by side against the acceptance bounds of the evaluation protocol and against their on-disk footprints, and identifies the configuration with the smallest exported footprint and the leading point-wise and structural scores.

5.1 Regression Stage

EDSR follows the smoothest of the three trajectories. The pixel loss falls from 0.08 at the start of training to $1.3 \cdot 10^{-3}$ at the end of one hundred fifty

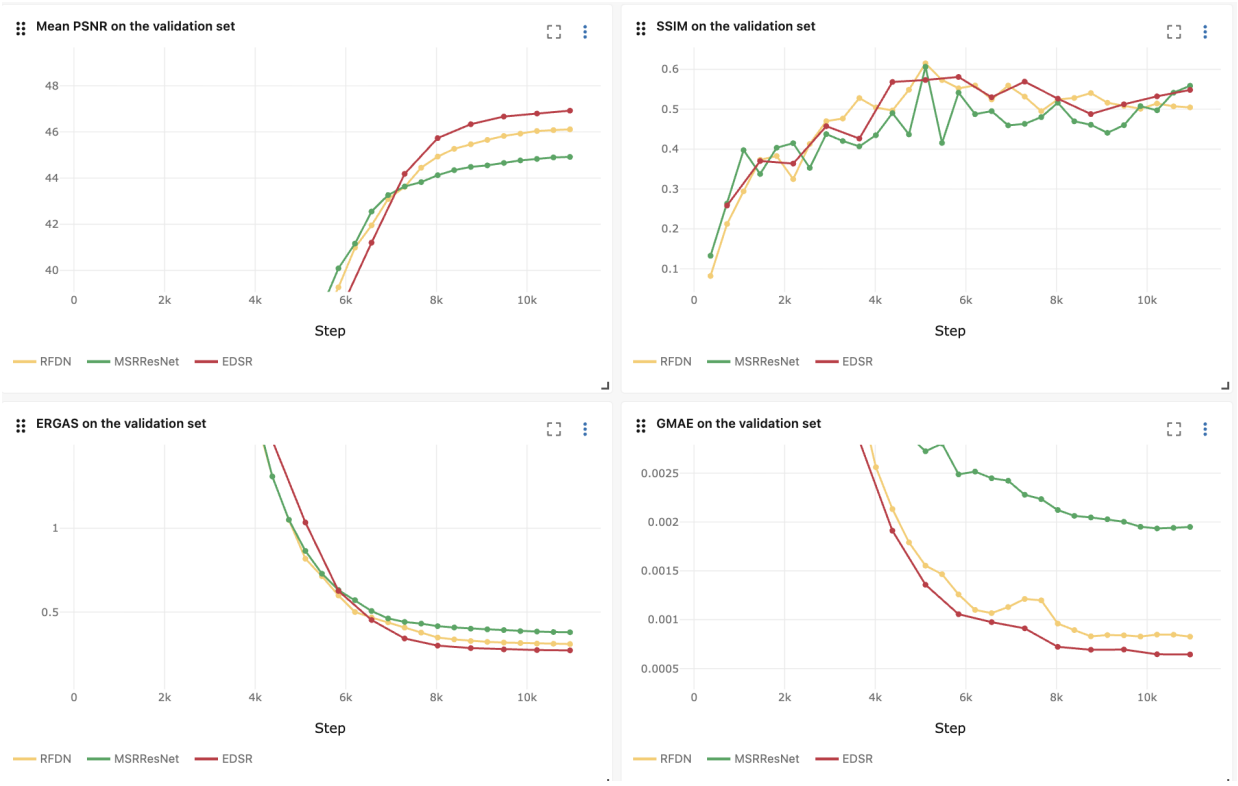

Fig. 5.1. Validation metrics over training epochs for the three generators in the regression stage.

epochs, with the gradient norm decaying from approximately 2 to 0.5 over the same horizon and no instability in between. The PSNR climbs from 7 dB at the tenth validation pass through the thirty-decibel threshold around epoch sixty to a final plateau near 46.8 dB; the channel-wise breakdown at the last epoch puts the pressure channel at 43.5 dB and the two saturation channels at 51.4 and 51.8 dB, consistent with the wider dynamic range of pressure. SSIM behaves differently: it rises to a peak of 0.63 near epoch eighty, drifts down to 0.44 by epoch one hundred thirty, and recovers only partially to 0.55 at the end. The drift away from the SSIM peak coincides with the second half of the regression schedule, where the pixel loss continues to fall but the structure of the displacement fronts begins to smooth out under the unrestrained L_1 objective.

MSRResNet converges more slowly and to a lower ceiling. The first five

validation passes leave the network essentially untrained, with the validation loss still near 0.8 and PSNR around one decibel, against the seven decibels reached by the deeper network at the same milestone. From epoch ten onward the trajectory normalizes, the training loss settles into the same $0.08 \rightarrow 1.4 \cdot 10^{-3}$ band, and the gradient norm contracts from 2.0 to 0.15. Final PSNR reaches 44.94 dB and the channel split is 42.2 / 46.9 / 49.1 dB for *P*, *S*, and $\bar{S}$, with the pressure channel trailing the saturations as in the larger backbone. SSIM shows the same regression-stage instability, swinging between 0.42 and 0.61 over neighboring validation passes around epoch seventy and finishing at 0.56 after a long plateau in which PSNR adds barely more than two decibels across fifty epochs, signaling that the capacity of the network has been reached.

RFDN occupies an intermediate position despite the smallest parameter count. Its training loss follows the curve $0.09 \rightarrow 1.1 \cdot 10^{-3}$ with a gradient norm that contracts from 1.6 to 0.07, the cleanest of the three. PSNR starts slowly, sits at 4.6 dB by epoch four, and converges by the tenth epoch to a trajectory that ends at 46.11 dB at the final checkpoint, 1.2 dB above MSRResNet and 0.7 dB below EDSR at a fraction of the parameter budget. The channel breakdown 43.1 / 49.5 / 51.1 dB for *P*, *S*, and $\bar{S}$ tracks EDSR closely, which suggests that the spatial attention block compensates for the narrower feature width on the saturation channels. SSIM peaks at 0.61 around epoch seventy and drifts down to 0.50 at the final checkpoint, repeating the structural-quality drop seen on the other two trajectories.

Figure 5.1 overlays the three runs on common axes for PSNR, SSIM, ERGAS, and GMAE. The shared pattern is a decoupling between the pixel objective and the structural metric in the second half of the schedule: PSNR continues to rise, the pixel loss continues to fall, and SSIM moves in the opposite direction by

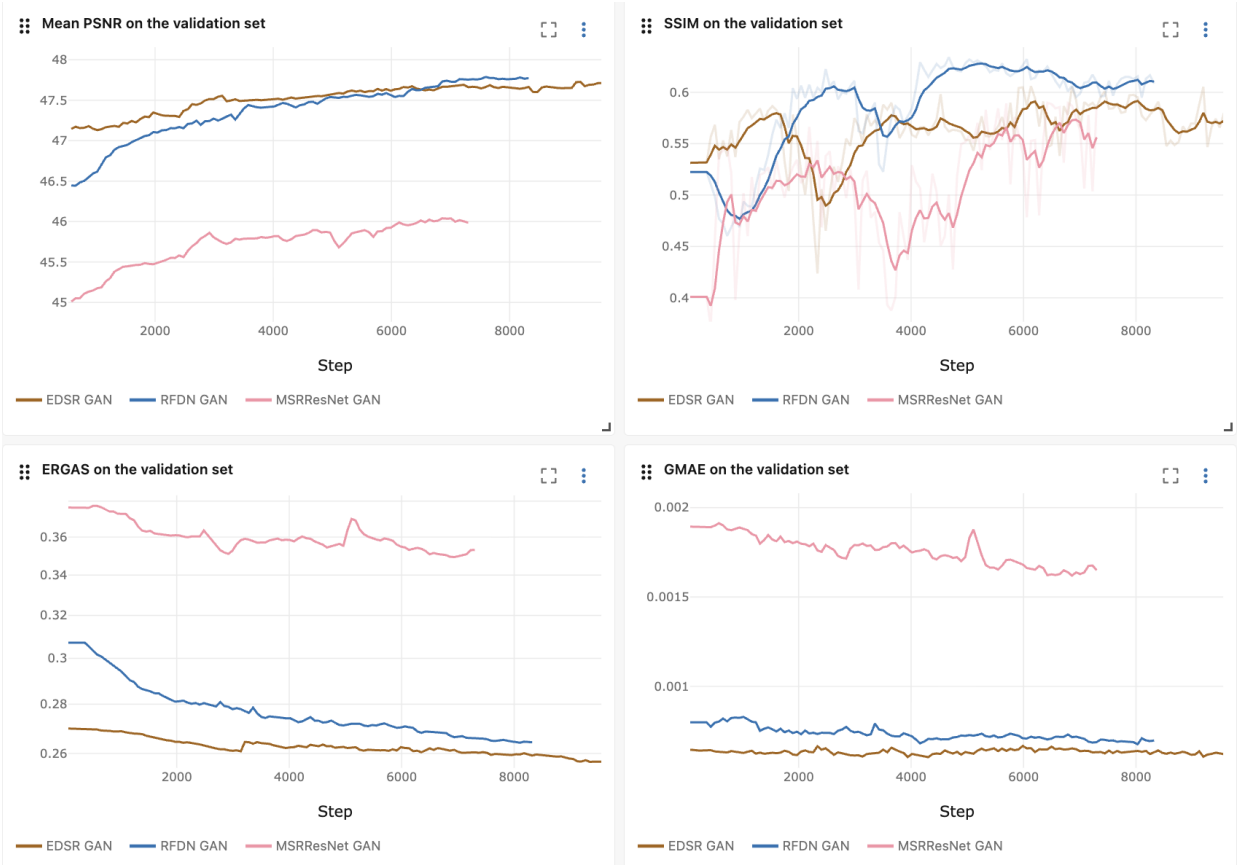

Fig. 5.2. Validation metrics over training epochs for the three generators in the adversarial stage.

between 0.06 and 0.11 points relative to its peak. The peak SSIM is reached on every backbone around epoch seventy to eighty, well before the final checkpoint, and is the natural reference point for the warm start of the adversarial stage.

5.2 Adversarial Stage

EDSR follows the cleanest of the three adversarial trajectories. The five-epoch discriminator warmup settles its loss near $\ln 2 \approx 0.69$, the five-epoch generator warmup on the pixel and perceptual terms raises SSIM from 0.53 to 0.57 without touching PSNR, and the full composite objective then opens with a textbook relativistic equilibrium: the critic-to-generator loss ratio sits at one

and the logit gap at zero for nine epochs. From epoch nineteen onward the discriminator starts to separate real residuals from fakes, the logit gap grows to 0.05, and the adversarial term begins to deliver gradient. The structural score reaches a first local peak of 0.60 at epoch forty-one and a global peak of 0.61 at epoch ninety-three; PSNR climbs to 47.71 dB at epoch one hundred thirty and the channel split at that point is 44.3 / 53.5 / 52.3 dB for P, S, and $\bar{S}$. From epoch fifty-six the skip rule starts to fire whenever the critic becomes too confident on the current batch, which keeps it from overwhelming the generator and explains the absence of any sign of mode collapse over the full one hundred thirty epochs.

MSRResNet yields the largest relative SSIM gain together with the most volatile training dynamics. The warmup phases follow the same recipe and lift SSIM from 0.40 to 0.54 on the perceptual term alone before the discriminator gradient is enabled. The full composite objective then drives the critic into a sequence of swings: at epoch thirty-three its loss briefly rises to 0.77, then collapses to $\ln 2$ for one validation window before recovering; at epoch sixty-eight the critic-to-generator ratio peaks at 1.54 and the skip rule activates for two consecutive epochs; from epoch ninety-three onward the rule fires intermittently as the discriminator keeps tightening on the smaller generator. PSNR rises to 46.04 dB at epoch ninety-three and stabilizes at 45.99–46.04 dB afterwards; SSIM oscillates with an amplitude of about ± 0.10 between epochs but reaches a peak of 0.59 at epochs eighty-seven, ninety-two, and ninety-five. The final channel split is 43.4 / 48.3 / 50.0 dB for P, S, and $\bar{S}$, with the pressure channel improving by one decibel relative to the pixel-only baseline.

RFDN gives the most stable trajectory of the three and reaches the highest PSNR and SSIM. The skip rule never activates over one hundred thirteen epochs, the critic-to-generator ratio stays in a narrow band around one outside a single

excursion to 1.63 at epoch forty-five, and both networks recover from that excursion without leaving any trace on the generator output. SSIM transiently dips from 0.52 to 0.46 during the generator warmup as the perceptual term reshapes the structural content, returns to the baseline by epoch eighteen, and then climbs to a peak of 0.629 at epoch fifty-six with further peaks above 0.62 at epochs sixty and sixty-one. PSNR rises steadily to 47.77 dB at the end of training, the highest of the three adversarial runs, and the channel split 44.6 / 51.8 / 52.8 dB is the most balanced as well, with the pressure channel landing 1.5 dB above the corresponding pixel-only run.

Figure 5.2 places the three runs on common axes for PSNR, SSIM, ERGAS, and GMAE. The shared pattern across the three trajectories is that the warmup decouples the pixel and the perceptual updates, the full composite objective then activates without an early SSIM collapse, and the skip rule contains the discriminator when it becomes overconfident on the current batch. PSNR rises by between one and one and a half decibels relative to the pixel-only baseline on every backbone, SSIM rises by between 0.07 and 0.13 points, and no run shows the trade-off between point-wise and structural quality reported in classical SRGAN training.

Summary

Table 5.1 brings together the validation scores of the six runs against the acceptance bounds of the evaluation protocol. A run is counted as accepted when all four scores meet their bounds simultaneously. None of the three pixel-only runs passes the full suite. MSRResNet sits 0.08 dB below on PSNR, EDSR falls 0.002 short on SSIM, and RFDN falls 0.045 below on the same metric.

TABLE 5.1
Validation metrics against acceptance bounds.

Configuration Acceptance bound	PSNR↑ ≥ 45	SSIM↑ ≥ 0.55	GMAE↓ ≤ 0.002	ERGAS↓ ≤ 0.4
Regression stage				
MSRResNet	44.92	**0.559**	0.00195	0.380
EDSR	**46.92**	0.548	**0.00064**	**0.272**
RFDN	46.11	0.505	0.00083	0.310
Adversarial fine-tune				
MSRResNet GAN	45.99	0.590	0.00165	0.353
EDSR GAN	47.71	0.606	**0.00062**	**0.257**
RFDN GAN	**47.77**	**0.634**	0.00070	0.264

The point-wise scores in the regression stage are otherwise comfortable on all three backbones: GMAE stays between three and thirty times under its bound and ERGAS between one and two orders of magnitude under. The picture is consistent with the regression-stage trajectories analyzed earlier in this chapter: the pixel objective alone drives PSNR well past the required level on the two deeper backbones but cannot lift SSIM to that level on the displacement fronts.

The adversarial fine-tune closes the gap on every architecture. All three GAN runs pass all four bounds, with margins between 0.04 and 0.08 on SSIM and between 1.0 and 2.8 dB on PSNR. The relative SSIM gain over the corresponding pixel-only run is largest for RFDN ($+0.129$ from 0.505), which goes from the only backbone that misses three bounds to the configuration with the best point-wise and structural scores, leading on PSNR and SSIM. PSNR rises by between 1.0 and 1.7 dB across the three runs, against the conventional trade-off reported for classical adversarial super-resolution in which PSNR is sacrificed for perceptual quality. The composite objective with the dual-porosity physics priors and the discriminator skip rule preserves the point-wise accuracy reached during pixel-

TABLE 5.2
Model sizes.

Architecture	Params (M)	Regression checkpoint (MB)	Adversarial checkpoint (MB)	ONNX (MB)
RFDN	0.79	12.31	65.66	3.07
MSRResNet	0.93	14.22	67.53	3.55
EDSR	1.52	23.29	76.61	5.81

only training and adds the structural gain on top.

Model sizes are collected in Table 5.2. Parameter counts span a factor of two across the three architectures, from 0.79 million for RFDN to 1.52 million for EDSR. The PyTorch checkpoint of an adversarial run is roughly four to five times heavier than the regression-stage checkpoint of the same architecture, because the file on disk there carries the discriminator and the moving-average copy of the generator in addition to the generator itself. The exported ONNX artifact, in contrast, contains only the moving-average generator weights, so its size scales with the architecture alone and does not change between the two training stages. ONNX artifacts range from 3.07 MB for RFDN to 5.81 MB for EDSR; the larger PyTorch files belong to the development workflow and are not deployed.

RFDN GAN combines the smallest exported footprint with the leading PSNR and SSIM scores in the suite, while EDSR GAN attains the best GMAE and ERGAS. EDSR GAN trails RFDN GAN on SSIM by 0.028 points at almost twice the parameter count. MSRResNet GAN passes every bound and is the lightest backbone among those that fail in the pixel-only regime, which makes it the natural candidate for resource-constrained deployment if RFDN is not available.

Chapter 6

Conclusion

This thesis developed a neural network model for the reconstruction of pressure and saturation fields in fractured-porous reservoirs from coarse-grid numerical solutions. A dataset of paired coarse and fine simulations was generated using Latin Hypercube Sampling (LHS) over the parameters of the dual-porosity model. Three residual super-resolution backbones (MSRResNet, EDSR, RFDN) were trained with a pixel-only loss and then fine-tuned with a combined loss that joins the pixel term with a domain-specific perceptual term, a reservoir-physics regularizer, and a relativistic adversarial term against a residual patch critic.

Under the pixel-only loss the reconstruction error decreases with the depth of the backbone, but the structural similarity score stops improving well before the parameter budget runs out, so the depth hypothesis holds for pixel accuracy and breaks down for structural quality. Adversarial fine-tuning then raises both the structural and the pixel scores at the same time across all three backbones, which goes against the usual trade-off in which one is given up for the other; the reservoir-physics regularizer and the discriminator skip rule together account for this by keeping the point-wise side in place while the critic works on the

high-frequency residual that the pixel loss cannot see.

Alongside these findings the work delivers a software stack ready for deployment: a Hydra-configured training pipeline produces Lightning checkpoints, which an export step converts into verified ONNX files for use by a desktop application built on PySide6 and ONNX Runtime that places the neural reconstruction next to either the output of the numerical solver or a stored fine-grid reference.

The scope of these conclusions is narrow. The spatial domain is restricted to a two-dimensional axisymmetric section, which leaves out lateral changes not aligned with the radial axis, and the training pool comes from a small LHS campaign, so the coverage of the parameter space is thin and is a likely reason for the structural-quality ceiling observed under the pixel loss alone. Because the coarse-grid input is an independent numerical solution rather than a downsampling of the target, the trained generator learns to undo the smoothing of the coarse-grid solver rather than a known interpolation kernel, which limits its transfer to simulators with very different discretization schemes.

A natural extension is a larger generation campaign, with more cases and wider coverage of the parameter space, to push the structural metric past the ceiling reached with the present budget. A second direction is to condition the generator on a vector of dynamic simulation parameters, since the same coarse-grid state can map to different fine-grid states under different operating regimes, and this auxiliary context may resolve the ambiguity that a purely spatial input leaves open. The trained generator can also be plugged into a closed-loop workflow that runs many simulations in parallel for uncertainty assessment, replacing the fine-grid solver in the inner loop and measuring the speedup at a fixed compute budget.

Bibliography cited

[1] International Energy Agency. "World energy investment 2024," Accessed: Nov. 26, 2025. [Online]. Available: https://www.iea.org/reports/world-energy-investment-2024

[2] N. G. Saleri and R. M. Toronyi, "Engineering control in reservoir simulation: Part I," in *SPE Annual Technical Conference and Exhibition*, Oct. 1988. DOI: 10.2118/18305-MS

[3] P. Ringrose and M. Bentley, *Reservoir Model Design: A Practitioner's Guide*. Springer Netherlands, 2015. DOI: 10.1007/978-94-007-5497-3

[4] J. D. Jansen, S. D. Douma, D. R. Brouwer, P. M. J. Van den Hof, O. H. Bosgra, and A. W. Heemink, "Closed-loop reservoir management," in *SPE Reservoir Simulation Conference*, Feb. 2009. DOI: 10.2118/119098-MS

[5] E. Kusimova, L. Saychenko, N. Islamova, P. Drofa, E. Safiullina, and A. V. Dengaev, "Application of machine learning methods for predicting well disturbances," *Journal of Applied Engineering Science*, vol. 21, no. 1, pp. 76–86, 2023. DOI: 10.5937/jaes0-38729

[6] A. Ghamdi, S. Ganis, and K. Hammad, "Evaluation of ensemble smoother with multiple data assimilation and evolutionary algorithm for history

matching process optimization," in *SPE Asia Pacific Oil and Gas Conference and Exhibition*, Nov. 2020. DOI: 10.2118/202216-MS

[7] M. E. Hayder et al., "Designing a high performance computational platform for simulation of giant reservoir models," in *SPE Middle East Oil and Gas Show and Conference*, Mar. 2013. DOI: 10.2118/164429-MS

[8] M. H. El Dabbour et al., "Cloud-based agile reservoir modelling enriched with machine learning improved the opportunity identification in a mature gas condensate," in *Abu Dhabi International Petroleum Exhibition and Conference*, Oct. 2022. DOI: 10.2118/211632-MS

[9] S. I. Aanonsen, G. Nævdal, D. S. Oliver, A. C. Reynolds, and B. Vallès, "The ensemble Kalman filter in reservoir engineering—a review," *SPE Journal*, vol. 14, no. 03, pp. 393–412, Sep. 2009. DOI: 10.2118/117274-PA

[10] M. A. Cardoso and L. J. Durlofsky, "Use of reduced-order modeling procedures for production optimization," *SPE Journal*, vol. 15, no. 02, pp. 426–435, Dec. 2010. DOI: 10.2118/119057-PA

[11] J. Pola, S. Geiger, E. Mackay, M. Bentley, C. Maier, and A. Al-Rudaini, "The impact of micro- and meso-scale heterogeneities on surfactant flooding in carbonate reservoirs," in *SPE Europec featured at 81st EAGE Conference and Exhibition*, Jun. 2019. DOI: 10.2118/195557-MS

[12] I. Konyukhov and V. Konyukhov, "Analysis of the effectiveness of super-resolution algorithms for improving computational results: A case study on two-dimensional nonlinear diffusion problems," *Mathematics*, vol. 14, no. 1, pp. 1–15, Jan. 2026, Preprint / Under Review.

[13] P. S. da Cruz and C. V. Deutsch, "Reservoir management decision-making in the presence of geological uncertainty," Ph.D. dissertation, Department of Petroleum Engineering, Stanford University, Stanford, California, USA, 2000.

[14] Subsurface Dynamics. "The ultimate guide to oil production forecasting: Methods and analysis," Accessed: Nov. 26, 2025. [Online]. Available: https://www.ssdynamics.com/the-ultimate-guide-to-oil-production-forecasting-methods-and-analysis/

[15] Subsurface AI. "Rule-based reservoir modeling: Capturing reservoir heterogeneity that matters most to fluid flow—part 1," Accessed: Nov. 27, 2025. [Online]. Available: https://subsurfaceai.ca/rule-based-reservoir-modeling-capturing-reservoir-heterogeneity-that-matters-most-to-fluid-flow-part-1/

[16] Studfile. "Types of reservoir models." (in Russian), Accessed: Nov. 27, 2025. [Online]. Available: https://studfile.net/preview/7795039/page:2/

[17] R. D. Kanevskaya, V. V. Kadet, and V. I. Selyakov, *Subsurface Hydromechanics. Hydrodynamics of Fractured-Porous Reservoirs*. Moscow-Izhevsk: Institute of Computer Science, 2014, (in Russian).

[18] V. R. Dushin, O. A. Logvinov, E. I. Skryleva, and A. A. Shamina, *Modeling of unstable displacement of liquids from porous media taking into account external effects aimed at increasing oil recovery*, Research Project No. 20-07-00378, Lomonosov Moscow State University, (in Russian), 2022. Accessed: Nov. 27, 2025. [Online]. Available: https://istina.msu.ru/projects/341406269/

[19] D. W. Peaceman, *Fundamentals of Numerical Reservoir Simulation*. Elsevier Science, 2000, vol. 6.

[20] V. M. Konyukhov, I. V. Konyukhov, and A. N. Chekalin, "Numerical simulation, parallel algorithms and software for performance forecast of the system "fractured-porous reservoir—producing well" during its commissioning into operation," *Computer Research and Modeling*, vol. 11, no. 6, pp. 1069–1075, 2019. DOI: 10.20537/2076-7633-2019-11-6-1069-1075

[21] V. M. Konyukhov, S. V. Krasnov, and I. V. Konyukhov, "Numerical and algorithmic models of implosion in the system oil reservoir—well—movable implosion chamber," *Automation, Telemechanization and Communication in Oil Industry*, no. 4, pp. 23–30, 2016, (in Russian).

[22] M. Mathieu, C. Couprie, and Y. LeCun, "Deep multi-scale video prediction beyond mean square error," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[23] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Global. Pearson Education, 2018.

[24] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. DOI: 10.1109/TIP.2003.819861

[25] L. Wald, *Data Fusion: Definitions and Architectures. Fusion of Images of Different Spatial Resolutions*. Paris: Presses des MINES, 2002, ISBN: 2-911762-38-X.

[26] F. A. Kruse et al., "The spectral image processing system (SIPS)—interactive visualization and analysis of imaging spectrometer data," *Remote Sensing of Environment*, vol. 44, no. 2–3, pp. 145–163, 1993. DOI: 10.1016/0034-4257(93)90013-N

[27] I. Sobel and G. Feldman, "A 3x3 isotropic gradient operator for image processing," in *Pattern Classification and Scene Analysis*, R. O. Duda and P. E. Hart, Eds., John Wiley & Sons, 1968, pp. 271–272.

[28] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. DOI: 10.1016/j.jcp.2018.10.045

[29] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a deep convolutional network for image super-resolution," in *Computer Vision – ECCV 2014*, Springer, 2014, pp. 184–199.

[30] W. Shi et al., *Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network*, 2016. arXiv: 1609.05158 [cs.CV].

[31] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[32] J. Kim, J. K. Lee, and K. M. Lee, "Accurate image super-resolution using very deep convolutional networks," in *Proceedings of the IEEE Conference*

on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 1646–1654.

[33] C. Ledig et al., "Photo-realistic single image super-resolution using a generative adversarial network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[34] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, pp. 136–144.

[35] Z. Hui, X. Gao, Y. Yang, and X. Wang, "Lightweight image super-resolution with information multi-distillation network," in *Proceedings of the 27th ACM International Conference on Multimedia (ACM MM)*, 2019, pp. 2024–2032. DOI: 10.1145/3343031.3351084

[36] J. Liu, J. Tang, and G. Wu, "Residual feature distillation network for lightweight image super-resolution," in *European Conference on Computer Vision Workshops (ECCVW)*, 2020. DOI: 10.1007/978-3-030-67070-2_2

[37] I. Goodfellow et al., "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014.

[38] X. Wang et al., "ESRGAN: Enhanced super-resolution generative adversarial networks," in *Proceedings of the European Conference on Computer Vision Workshops (ECCVW)*, 2018.

[39] A. Jolicoeur-Martineau, *The relativistic discriminator: A key element missing from standard GAN*, 2018. arXiv: 1807.00734 [cs.LG].

[40] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[41] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-k. Wong, and W.-c. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 802–810.

[42] J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte, *SwinIR: Image restoration using swin transformer*, 2021. arXiv: 2108.10257 [eess.IV].

[43] C. Saharia, J. Ho, W. Chan, T. Salimans, D. J. Fleet, and M. Norouzi, *Image super-resolution via iterative refinement*, 2021. arXiv: 2104.07636 [eess.IV].

[44] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, *High-resolution image synthesis with latent diffusion models*, 2022. arXiv: 2112.10752 [cs.CV].

[45] Z. Deng, C. He, Y. Liu, and K. C. Kim, "Super-resolution reconstruction of turbulent velocity fields using a generative adversarial network-based artificial intelligence framework," *Physics of Fluids*, vol. 31, no. 12, p. 125 111, 2019. DOI: 10.1063/1.5127031

[46] N. Doloi, S. Ghosh, and J. Phirani, "Super-resolution reconstruction of reservoir saturation map with physical constraints using generative adversarial network," in *SPE Reservoir Characterisation and Simulation Conference and Exhibition*, Jan. 2023. DOI: 10.2118/212611-MS

[47] B. M. Aslam, H. Li, and B. Yan, "Hybrid multiscale simulation enhanced by super-resolution for multiphase flow in high-contrast heterogeneous reservoirs," in *SPE Middle East Oil and Gas Show and Conference*, Sep. 2025. DOI: 10.2118/227634-MS

[48] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 42, no. 1, pp. 55–61, 2000. DOI: 10.1080/00401706.2000.10485979